# 第7章 树和二叉树

提纲

CONTENTS


7.1 树

7.2 二 叉 树

7.3 二叉树先序、中序和后序遍历

7.4 二叉树的层次遍历

7.5 二 叉 树的构造

7.6 线索二叉树

7.7 哈夫曼树

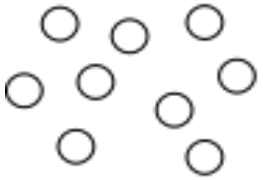
7.8 二叉树与树、森林之间的转换

7.9 并查集

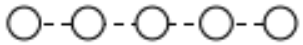
# 数据的逻辑结构

逻辑上，数据元素之间的关系只有4种：无关系、一对一关系、一对多关系、多对多关系，这4种逻辑关系总称为数据的逻辑结构。

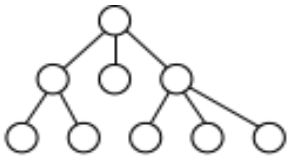
- 集合：包含的所有数据元素之间无关系，即数据元素之间的次序是任意的。



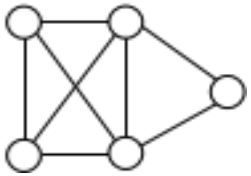
- 线性结构：包含的数据元素之间存在一对一的关系，即数据元素之间构成一个有序序列。



- 树形结构：包含的数据元素之间存在一对多的关系，即数据元素之间形成一个层次关系。



- 图形结构：包含的数据元素（结点）之间存在多对多的关系，即图中每个数据元素的前驱和后继数目都不限。



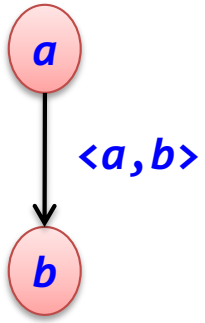
# 7.1 树

## 7.1.1 树的定义

- 树是由$n$（$n \geq 0$）个结点组成的有限集合（记为T）。
- 如果$n=0$，它是一棵空树，这是树的特例。
- 如果$n>0$，这$n$个结点中存在（有仅存在）一个结点作为树的根结点（root），其余结点可分为$m$（$m \geq 0$）个互不相交的有限集$T_1$、$T_2$、…、$T_m$，其中每个子集本身又是一棵符合本定义的树，称为根结点的子树。

树是一种非线性数据结构，具有以下特点：

- 每一结点可以有零个或多个后继结点，但有且只有一个前驱结点（根结点除外）。
- 数据结点按分支关系组织起来，清晰地反映了数据元素之间的层次关系。

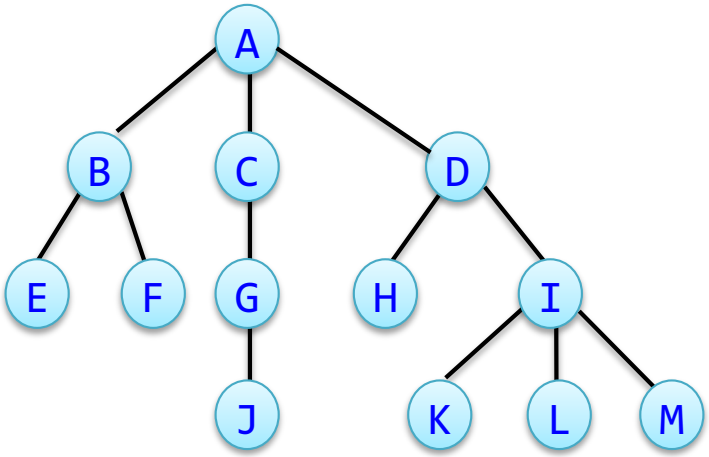


## 抽象数据类型树的描述

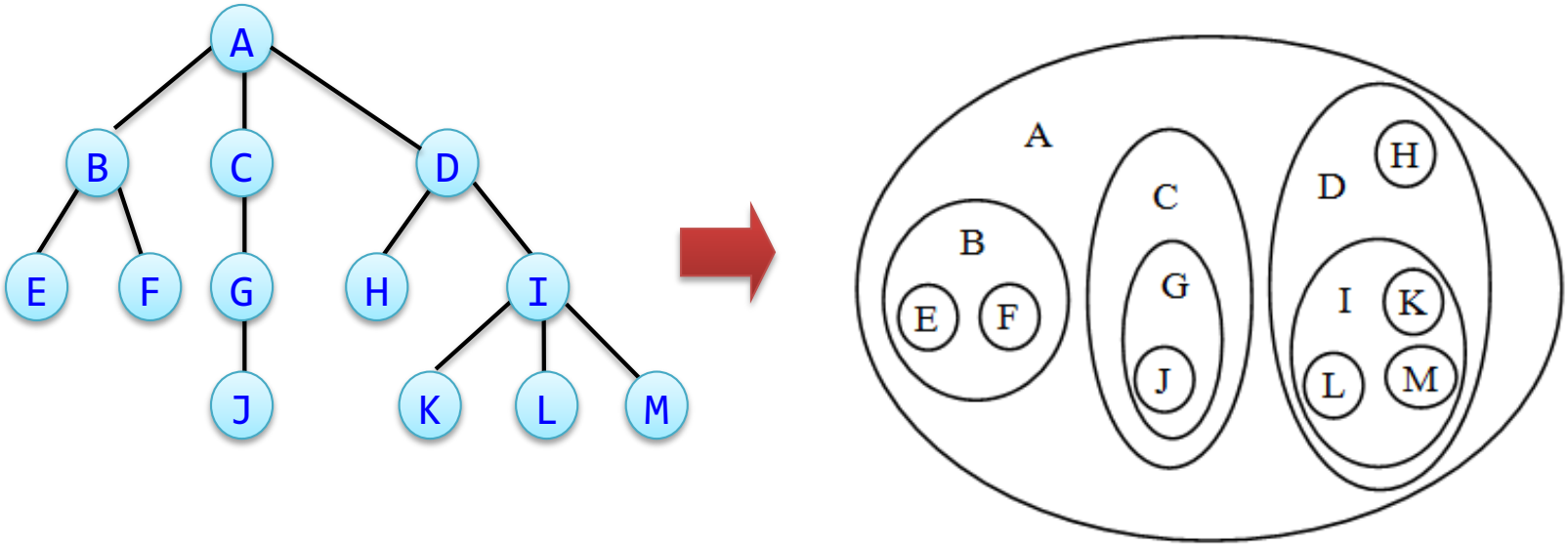
ADT Tree
{
数据对象：
D={$a_i$ | 0≤$i$≤$n$-1，$n$≥0，$a_i$为E类型}
数据关系：
R={$r$}
r={<$a_i$，$a_j$> | $a_i$，$a_j$∈D， 0≤$i$，$j$≤$n$-1，其中每个结点最多只有一个前驱结点、可以有零个或多个后继结点，有且仅有一个结点即根结点没有前驱结点 }
基本运算：
CreateTree()：由树的逻辑结构表示建立其存储结构。
DispTree()：输出树的括号表示串。
E GetParent(int $i$)：求编号为$i$的结点的双亲结点值。
…
}

# 7.1.2 树的逻辑结构表示方法

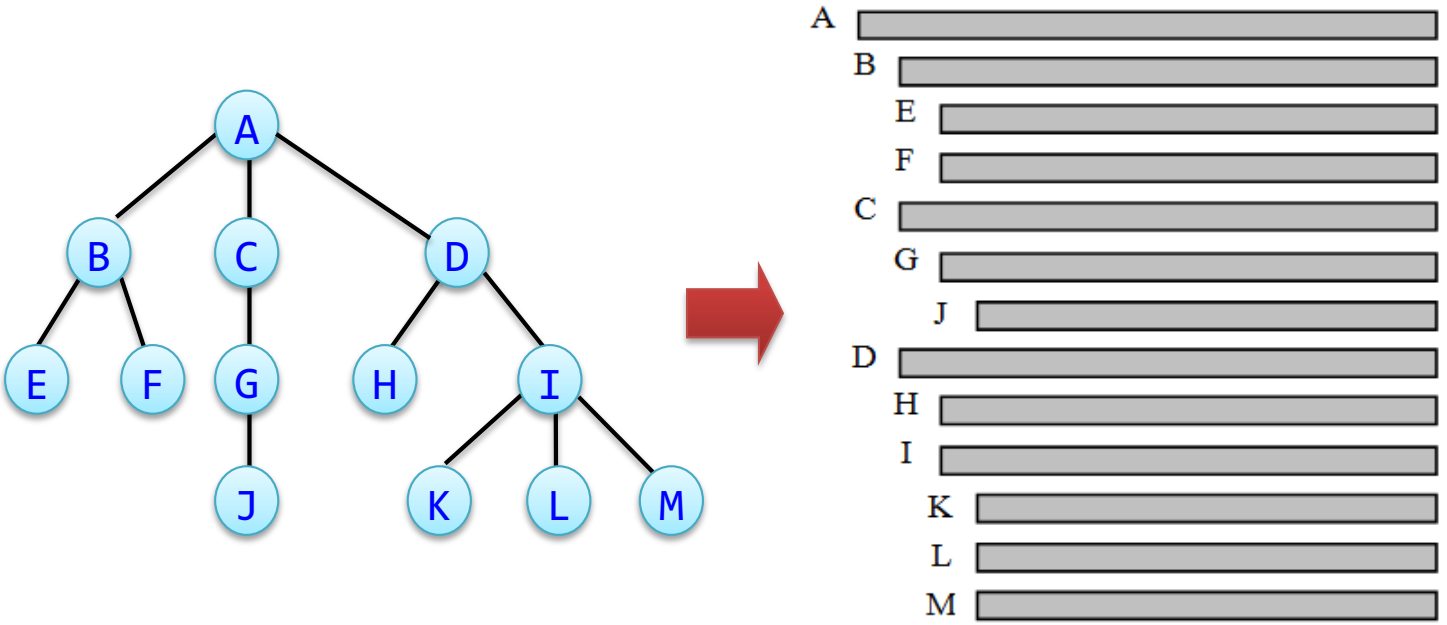
- **树形表示法**。这是树的最基本的表示，使用一棵倒置的树表示树结构，非常直观和形象。



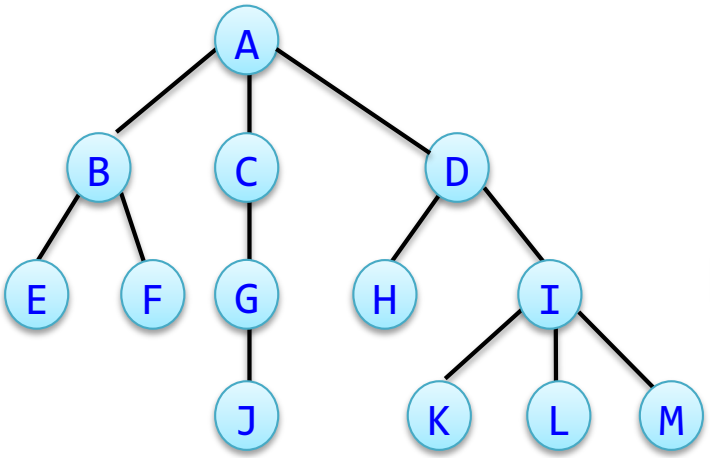
- **文氏图表示法**。使用集合以及集合的包含关系描述树结构。



- **凹入表示法**。使用线段的伸缩关系描述树结构。



- **括号表示法**。将树的根结点写在括号的左边，除根结点之外的其余结点写在括号中并用逗号分隔。

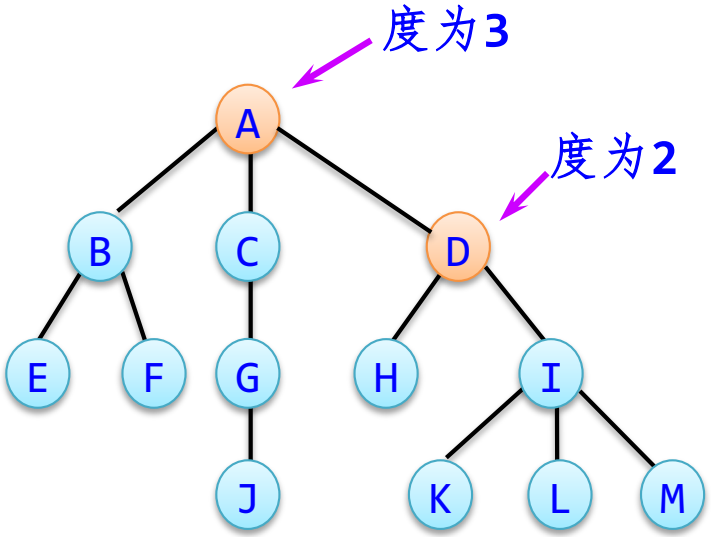



A(B(E,F),C(G(J)),D(H,I(K,L,M)))

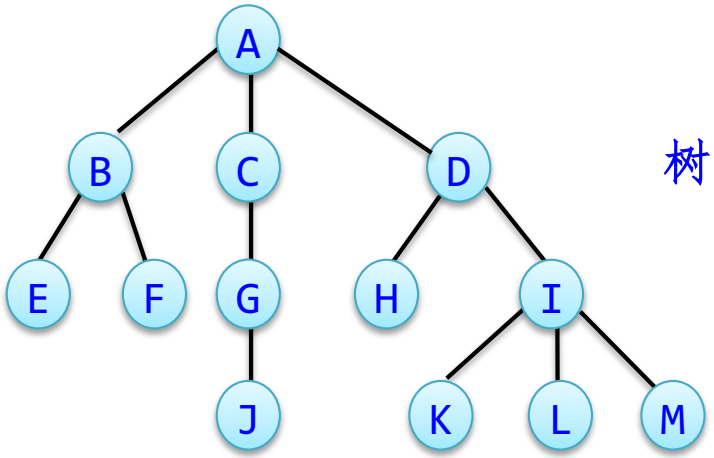
根(子树1，子树2，…，子树*m*)

# 7.1.3 树的基本术语

- **结点的度**。树中每个结点具有的子树数或者后继结点数称为该结点的度。

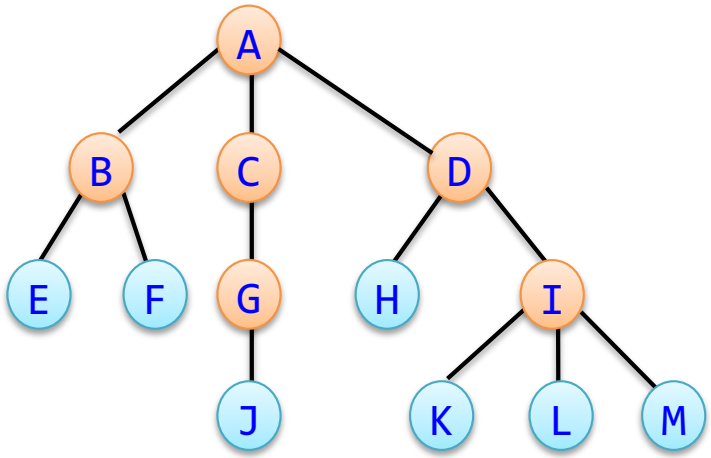


- **树的度**。树中所有结点的度的最大值称之为树的度。



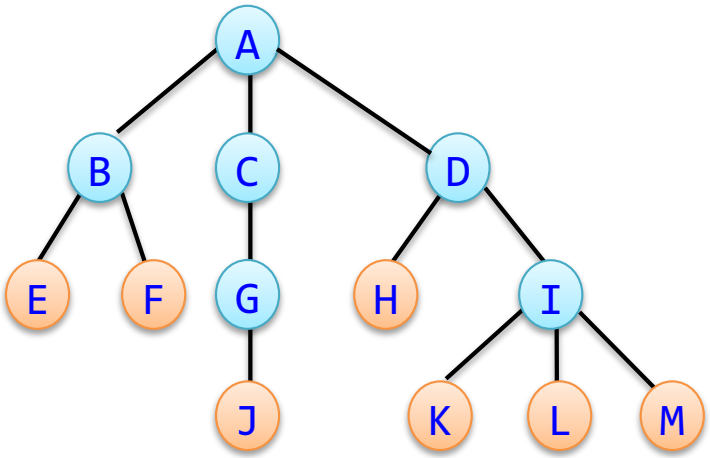

树的度为3

- **分支结点**。度大于0的结点称为分支结点或非终端结点。度为1的结点称为单分支结点，度为2的结点称为双分支结点，依次类推。



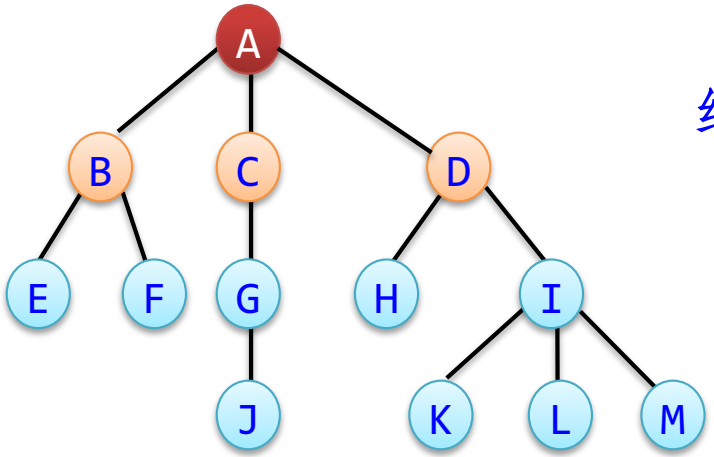

A、B、C、D、G、I为分支结点

- **叶子结点（或叶结点）。**度为零的结点称为叶子结点或终端结点。



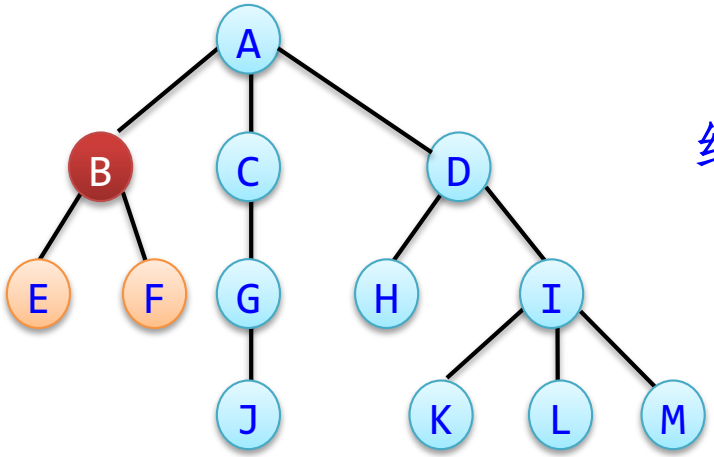

**E、F、J、H、K、L、M为叶子结点**

- **孩子结点**。一个结点的后继称之为该结点的孩子结点。



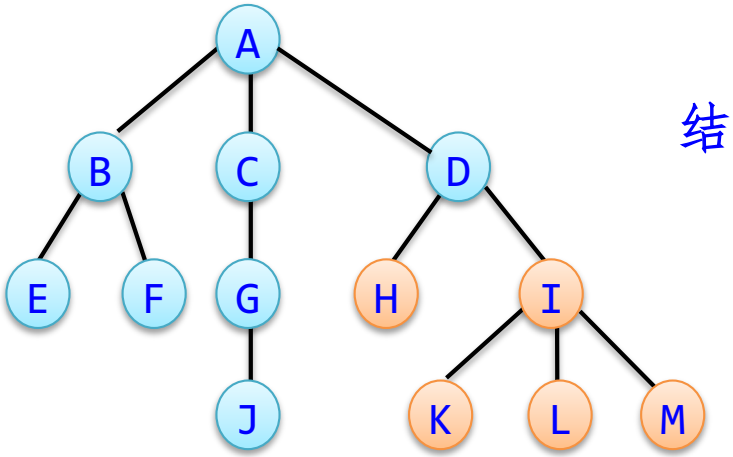

结点**A**的孩子结点为**B**、**C**和**D**

- **双亲结点（或父亲结点）**。一个结点称为其后继结点的双亲结点。



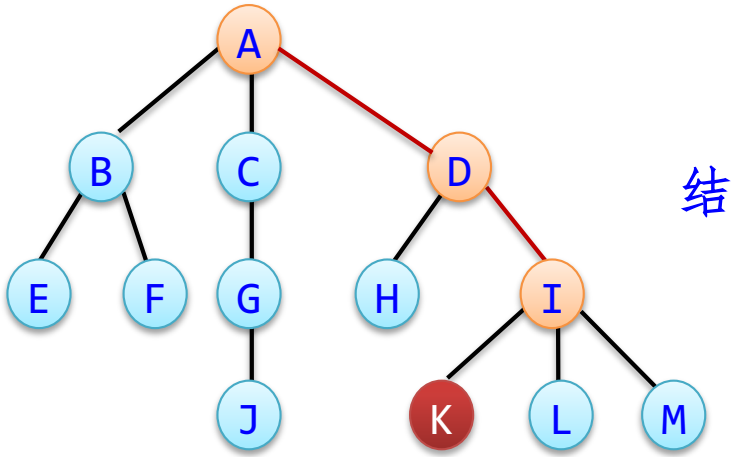

结点**E**和**F**的双亲结点均为**B**

- **子孙结点**。一个结点的子树中除该结点外的所有结点称之为该结点的子孙结点。



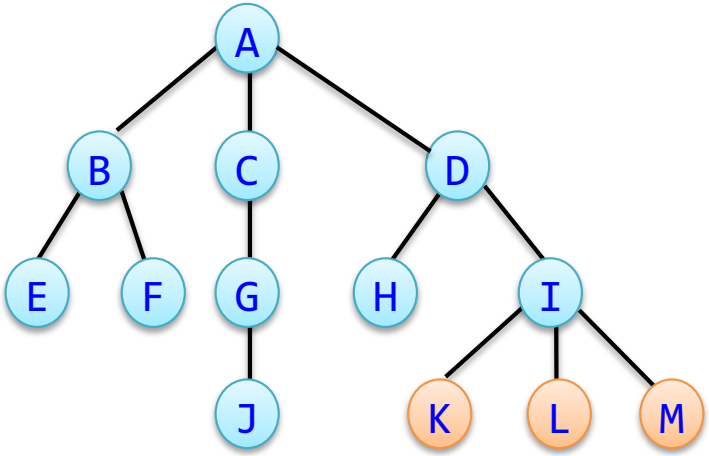

结点D结点的子孙结点为H、I、K、L、M

- **祖先结点**。从树根结点到达某个结点的路径上通过的所有结点称为该结点的祖先结点（不含该结点自身）。



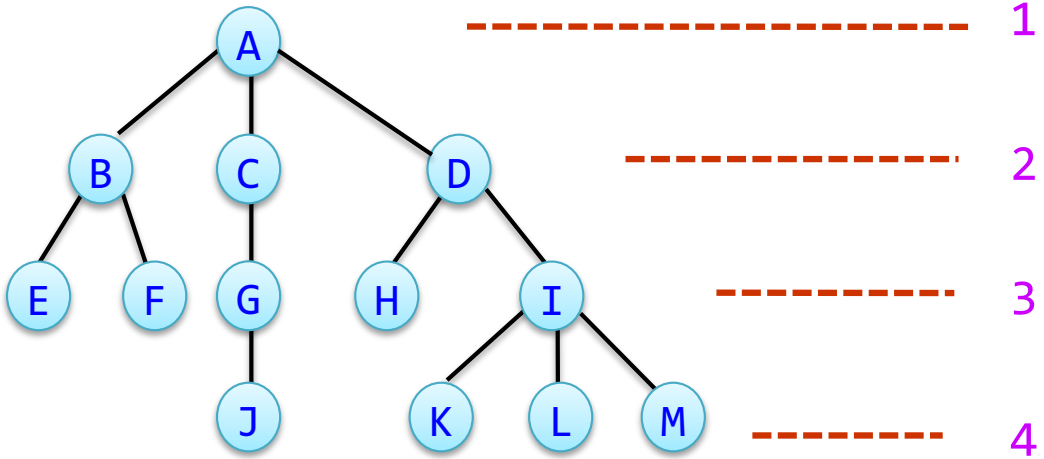

结点K的祖先结点为A、D、I

- **兄弟结点**。具有同一双亲的结点互相称之为兄弟结点。

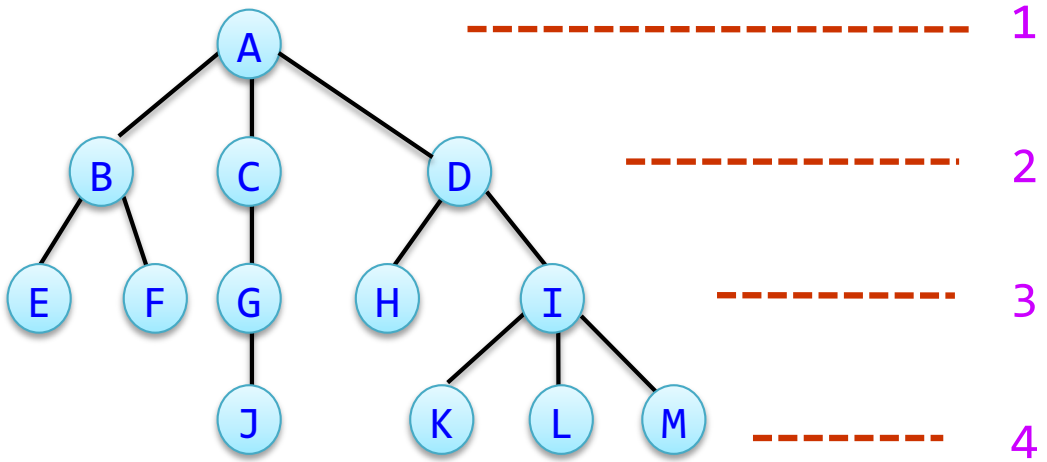



结点K、L、M是兄弟结点

- **结点层次**。树具有一种层次结构，根结点为第一层，其孩子结点为第二层，如此类推得到每个结点的层次。

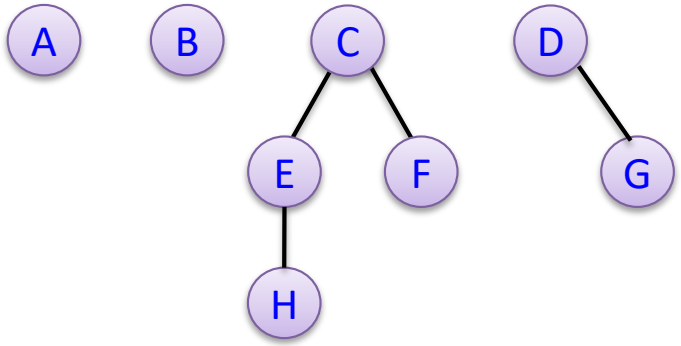


- **树的高度。**树中结点的最大层次称为树的高度或深度。




**高度是4**

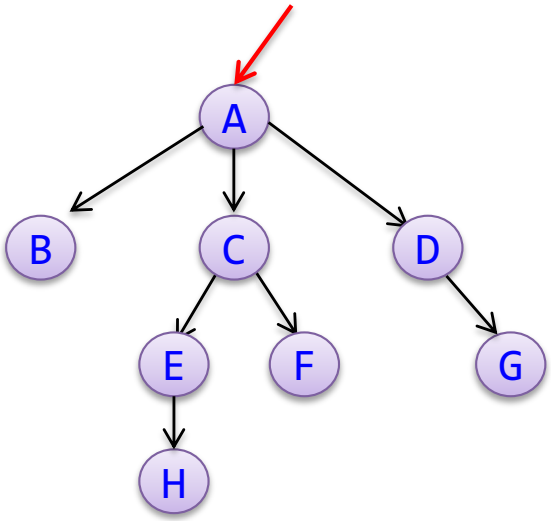
- **森林**。零棵或多棵互不相交的树的集合称为森林。




4棵树构成的森林

# 7.1.4 树的性质

**性质1：树中的结点数等于所有结点的度数加1。**




- 度之和=分支数
- 分支数=$n-1$
- 所以，$n$=度之和+1

**性质2**：度为$m$的树中第$i$层上至多有$m^{i-1}$个结点，这里应有$i \geq 1$。

**数学归纳法证明**

推广

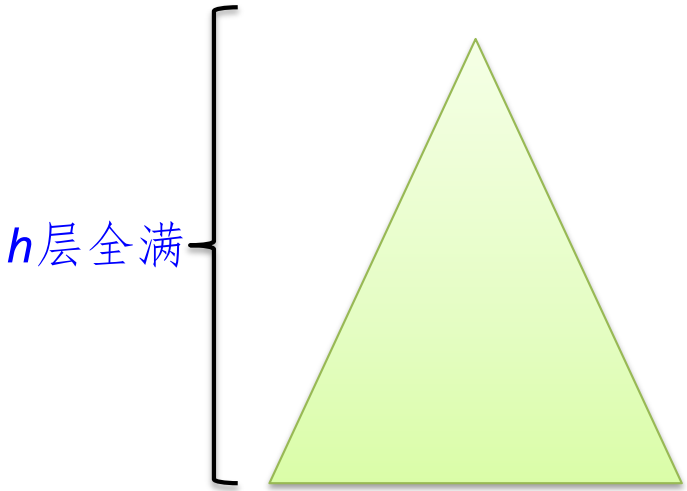
- 当一棵$m$次树的第$i$层有$m^{i-1}$个结点（$i \geq 1$）时，称该层是满的，若一棵$m$次树的所有叶子结点在同一层，每一层都是满的，称为满$m$次树。显然，满$m$次树是所有相同高度的$m$次树中结点总数最多的树。
- 也可以说，对于$n$个结点，构造的$m$次树为满$m$次树或者接近满$m$次树，此时树的高度最小。

**性质3：** 高度为$h$的$m$次树至多有 $\frac{m^h-1}{m-1}$ 个结点。

**由性质2推出**

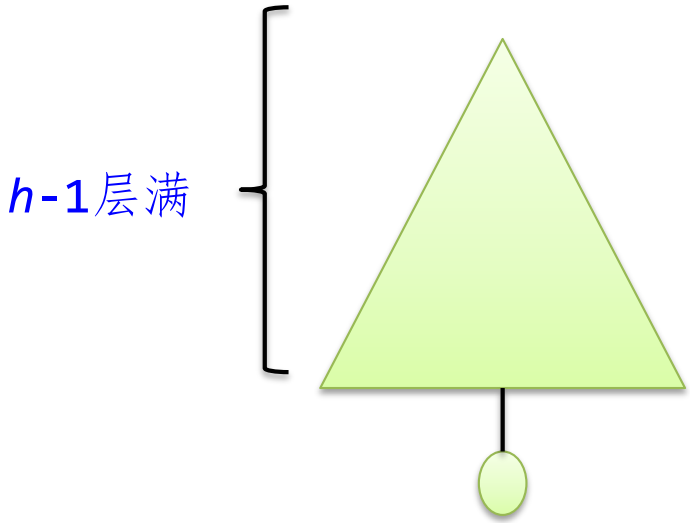
**性质4：**具有$n$个结点的$m$次树的最小高度为$\lceil \log_m(n(m-1)+1) \rceil$。

**证明：**设具有$n$个结点的$m$次树的最小高度为$h$，若在该树中前$h-1$层都是满的，即每一层的结点数都等于$m^{i-1}$个（$1 \leqslant i \leqslant h-1$），第$h$层（即最后一层）的结点数可能满，也可能不满，则该树具有最小的高度。其高度$h$可计算如下：




高度为$h$，结点个数最多的情况

$$\frac{m^h-1}{m-1}$$




高度为$h$，结点个数最少的情况

$$\frac{m^{h-1}-1}{m-1}+1$$

根据树的性质3可得：$\frac{m^{h-1}-1}{m-1} < n \leqslant \frac{m^{h}-1}{m-1}$

乘$(m-1)$后得：$m^{h-1} < n(m-1)+1 \leqslant m^{h}$

以$m$为底取对数后得：$h-1 < \log_m(n(m-1)+1) \leqslant h$

即 $\log_m(n(m-1)+1) \leqslant h < \log_m(n(m-1)+1)+1$

因$h$只能取整数，所以$h=\lceil \log_m(n(m-1)+1) \rceil$，结论得证。

【例7.1】 若一棵三次树中度为3的结点为2个，度为2的结点为1个，度为1的结点为2个，则该三次树中总的结点个数和度为0的结点个数分别是多少？

- 设该三次树中总结点个数、度为0的结点个数、度为1的结点个数、度为2的结点个数和度为3的结点个数分别为$n$、$n_0$、$n_1$、$n_2$和$n_3$。
- 显然，每个度为$i$的结点在所有结点的度数之和中贡献$i$个度。依题意有：$n_1=2$，$n_2=1$，$n_3=2$。由树的性质1可知

$$n = \text{所有结点的度数之和}+1$$
$$= 0\times n_0+1\times n_1+2\times n_2+3\times n_3+1$$
$$= 1\times 2+2\times 1+3\times 2+1=11$$

- 又因为$n=n_0+n_1+n_2+n_3$，即：$n_0=n-n_1-n_2-n_3=11-2-1-2=6$
- 所以该三次树中总的结点个数和度为0的结点个数分别是11和6。

【例7.2】已知一棵度为4的树中，度为$i$（$i\geq 1$）的结点个数有$i$个，问该树中有多少个叶子结点？

$n = n_0+n_1+n_2+n_3+n_4 = n_0+1+2+3+4 = n_0+10$，即：$n_0 = n-10$

度之和 $= n-1$

度之和 $= n_1+2n_2+3n_3+4n_4 = 30$

所以 $n = 30+1 = 31$，$n_0 = n-10 = 31-10 = 21$。

## 7.1.5 树的基本运算

树的运算主要分为三大类：

- 查找满足某种特定关系的结点，如寻找当前结点的双亲结点等；
- 插入或删除某个结点，如在树的当前结点上插入一个新结点或删除当前结点的第i个孩子结点等；
- 遍历树中每个结点。

- 树的遍历运算是指按某种方式访问树中的每一个结点且每一个结点只被访问一次。
- 有以下3种遍历方法：

  - 先根遍历
  - 后根遍历
  - 层次遍历

- **先根遍历:**

若树不空，则先访问根结点，然后依次先根遍历各棵子树。

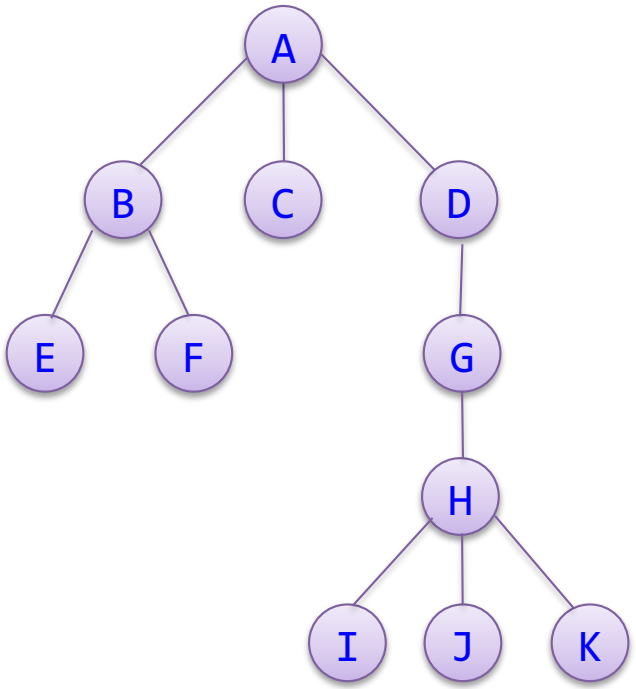
- **后根遍历:**

若树不空，则先依次后根遍历各棵子树，然后访问根结点。

- **层次遍历:**

若树不空，则自上而下自左至右访问树中每个结点。

**先根和后根遍历算法都是递归的。**




先根遍历的顶点访问次序：

A B E F C D G H I J K

后根遍历的顶点访问次序：

E F B C I J K H G D A

层次遍历的顶点访问次序：

A B C D E F G H I J K

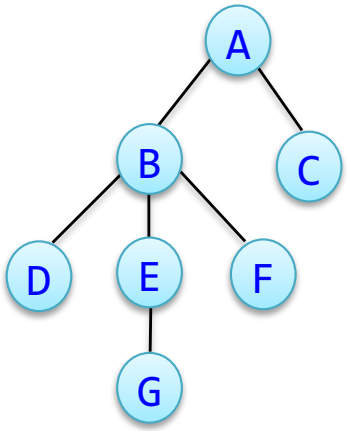
# 7.1.6 树的存储结构

## 1. 双亲存储结构

这种存储结构是一种顺序存储结构，用一组连续空间存储树的所有结点，同时在每个结点中附设一个伪指针指示其双亲结点的位置。

```
struct PNode                        //双亲存储结构元素类型
{   char data;                      //存放结点值,假设为char类型
    int parent;                     //存放双亲索引

    PNode(char d,int p)             //构造函数
    {   data=d;
        parent=p;
    }
};
```

```
vector<PNode> t;                    //树的双亲存储结构
```

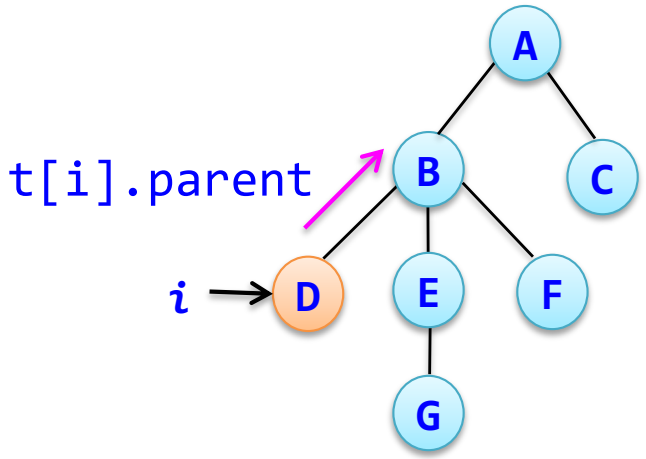



| 位置 | data | parent |
|---|---|---|
| 0 | A | -1 |
| 1 | B | 0 |
| 2 | C | 0 |
| 3 | D | 1 |
| 4 | E | 1 |
| 5 | F | 1 |
| 6 | G | 4 |

双亲存储结构：

- 利用了每个结点（根结点除外）只有唯一双亲的性质。
- 这种存储结构中，求某个结点的双亲结点十分容易，但求某个结点的孩子结点时需要遍历整个结构。

【例7.2】若一棵树采用双亲存储结构t存储，设计一个算法求指定索引是i的结点的层次。




| 位置 | data | parent |
|---|---|---|
| 0 | A | -1 |
| 1 | B | 0 |
| 2 | C | 0 |
| 3 | D | 1 |
| 4 | E | 1 |
| 5 | F | 1 |
| 6 | G | 4 |

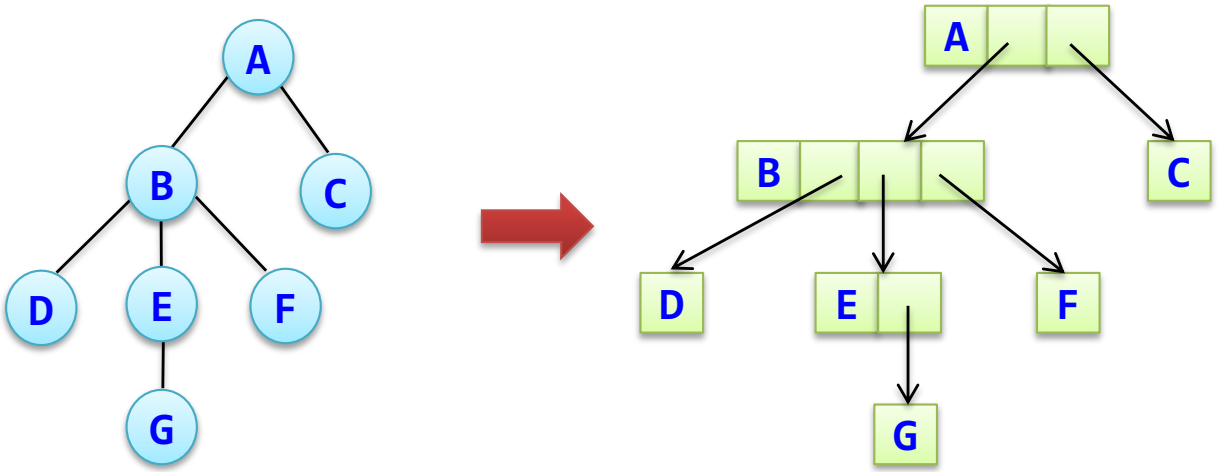
i →

SOLUTION

```
int Level(vector<PNode> t,int i)        //求t中索引i的结点的层次
{  if (i<0 || i>=t.size())              //参数错误返回0
      return 0;
   int cnt=1;
   while (t[i].parent!=-1)              //没有到达根结点时循环
   {  cnt++;
      i=t[i].parent;                    //移动到双亲结点
   }
   return cnt;
}
```

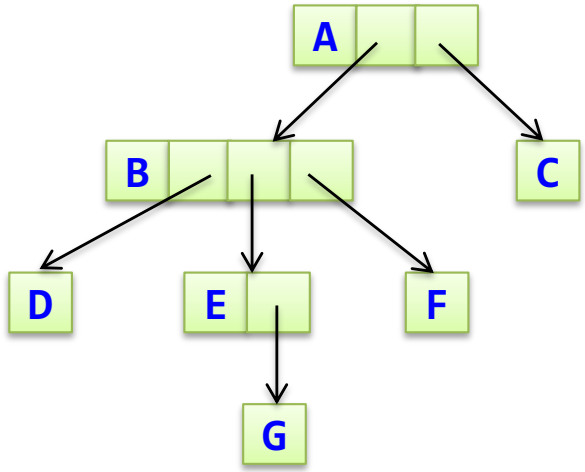
## 2．孩子链存储结构

每个结点包含结点值和所有孩子结点的指针，可按一个结点的度设计结点的孩子结点指针域个数。所有孩子结点指针用vector向量存储。



孩子链存储结构的结点类型SonNode定义如下：

```
struct SonNode                          //孩子链存储结构结点类
{   char data;                          //存放结点值,假设为char类型
    vector<SonNode* > sons;             //指向孩子结点指针的向量

    SonNode() {}                        //构造函数
    SonNode(char d):data(d) {}          //重载构造函数
};
```



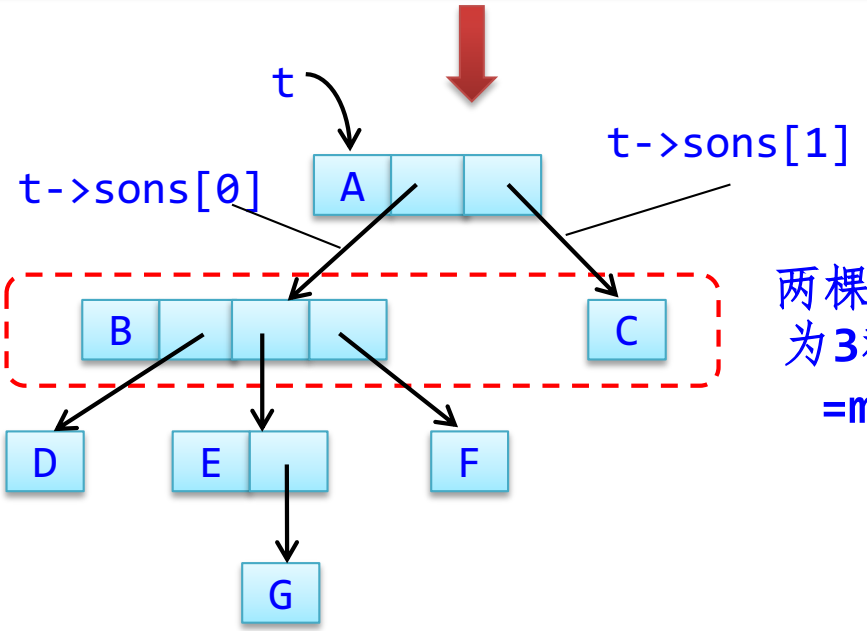
## 孩子链存储结构

- 优点是查找某结点的孩子结点十分方便。
- 缺点是查找某结点的双亲结点比较费时。

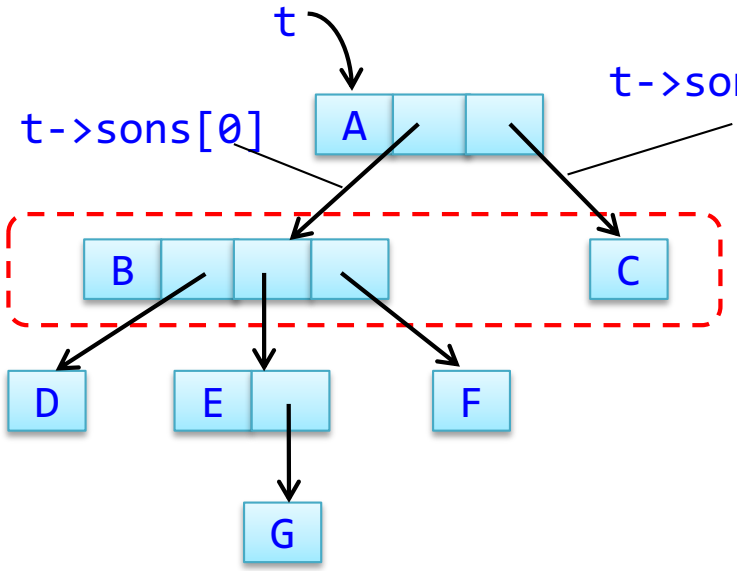
【例7.3】若一棵树采用孩子链存储结构t存储，设计一个算法求其高度。

一棵树的高度为根的所有子树高度的最大值加1。求整棵树的高度为“大问题”，求每棵子树高度为“小问题”。设$f(t)$为求树$t$的高度，对应的递归模型如下：

$$f(t)=0 \qquad \text{当}t\text{=NULL}$$
$$f(t)=max_i\, f(t->\text{sons}[i])+1 \qquad \text{其他情况}$$




两棵子树的高度分别为3和1，树$t$的高度
=max(3,1)+1=4




两棵子树的高度分别为3和1，树t的高度=max(3,1)+1=4

```
int Height(SonNode* t)                          //求t的高度
{  if (t==NULL)                                 //空树高度为0
      return 0;
   int maxsh=0;
   for (int i=0;i<t->sons.size();i++)           //遍历所有子树
   {  int sh=Height(t->sons[i]);                //求子树t->sons[i]的高度
      maxsh=max(maxsh,sh);                      //求所有子树的最大高度
   }
   return maxsh+1;
}
```

在一棵树T中最常用的操作是查找某个结点的祖先结点，采用**哪种存储结构最合适**？

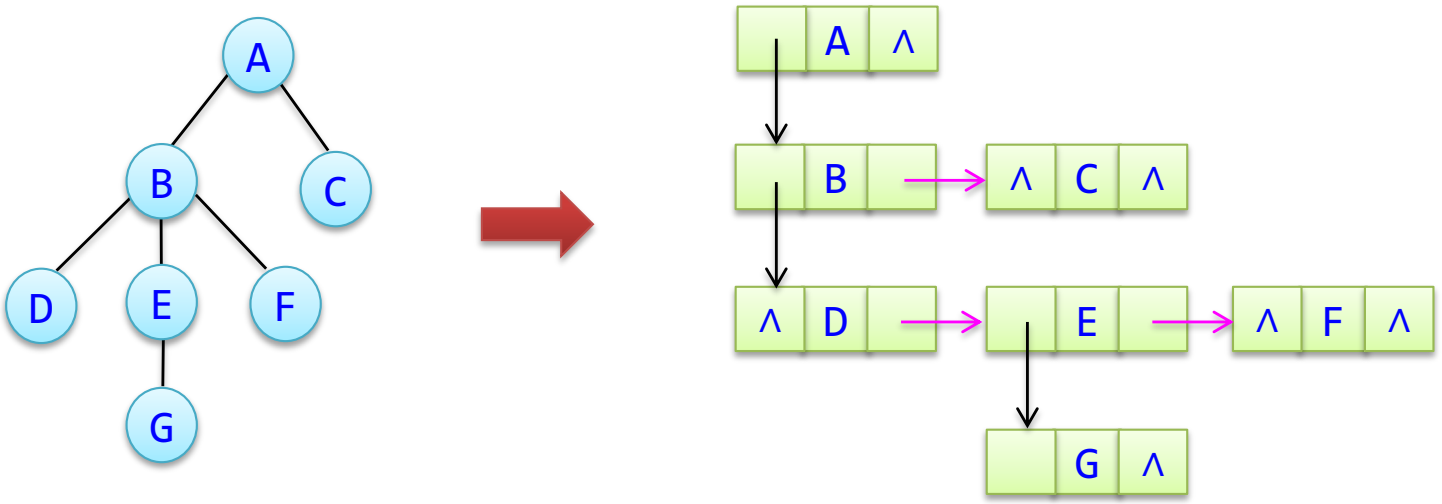
☑ 双亲存储结构

如最常用的操作是查找某个结点的所有兄弟，采用**哪种存储结构最合适**？

☑孩子兄弟链存储结构

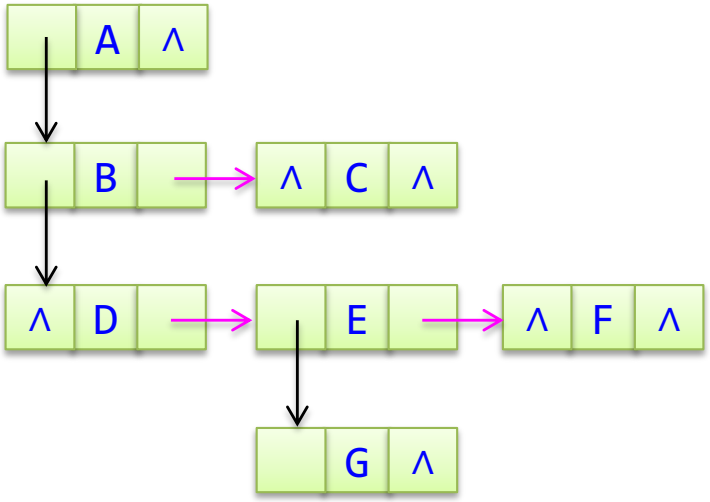
## 3．长子兄弟链存储结构

长子兄弟链存储结构是为每个结点设计三个域：一个数据元素域，一个指向该结点的长子的指针域，一个指向该结点的下一个兄弟结点指针域。



长子兄弟链存储结构中结点类型EBNode定义如下：

```
struct EBNode                                   //长子兄弟链中结点类
{   char data;                                  //结点的值
    EBNode* brother;                            //指向兄弟
    EBNode* eson;                               //指向长子结点

    EBNode():brother(NULL),eson(NULL) {}        //构造函数
    EBNode(char d)                              //重载构造函数
    {   data=d;
        brother=eson=NULL;
    }
};
```



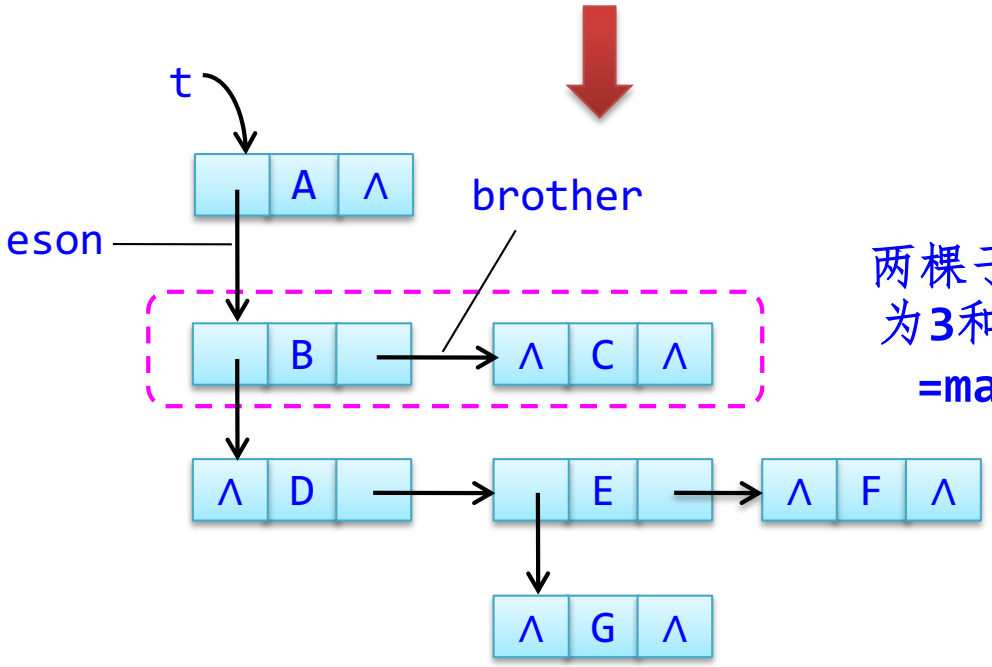
## 长子兄弟链存储结构

- 优点是查找某结点的孩子结点十分方便。
- 缺点是查找某结点的双亲结点比较费时。

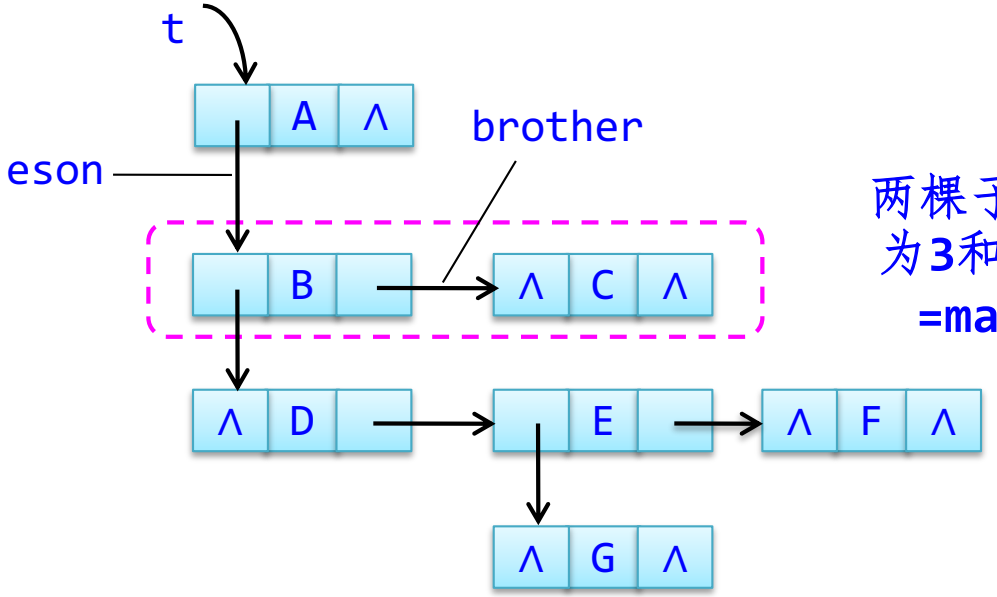
【例7.4】若一棵树采用长子兄弟链存储结构t存储，设计一个算法求其高度。

一棵树的高度为根的所有子树高度的最大值加1。求整棵树的高度为“大问题”，求每棵子树高度为“小问题”。设$f(t)$为求树$t$的高度，对应的递归模型如下：

$$f(t)=0 \qquad \text{当}t\text{=NULL}$$
$$f(t)=max_i\, f(t->\text{sons}[i])+1 \qquad \text{其他情况}$$




两棵子树的高度分别为3和1，树t的高度
=max(3,1)+1=4




两棵子树的高度分别为3和1，树t的高度=max(3,1)+1=4

```
int Height(EBNode* t)                    //求t的高度
{  if (t==NULL)                          //空树高度为0
      return 0;
   int maxsh=0;
   EBNode* p=t->eson;                    //p指向t结点的长子
   while (p!=NULL)
   {  EBNode* q=p->brother;              //q临时保存结点p的兄弟结点
      int sh=Height(p);                  //递归求结点p的子树的高度
      maxsh=max(maxsh,sh);               //求结点t的所有子树的最大高度
      p=q;
   }
   return maxsh+1;
}
```

# 练习题

为了实现以下各种功能，其中x结点表示该结点的位置，给出树的最适合的存储结构：

（1）求x和y结点的最近祖先结点。

（2）求x结点的所有子孙。

（3）求根结点到x结点的路径。

（4）求x结点的所有右边兄弟结点。

（5）判断x结点是否是叶子结点。

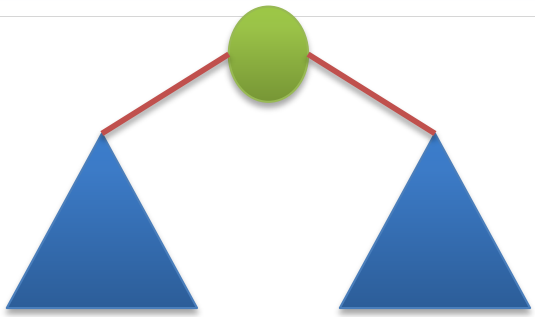
（6）求x结点的所有孩子。

# 7.2 二叉树

## 7.2.1 二叉树的概念

### 1. 二叉树的定义

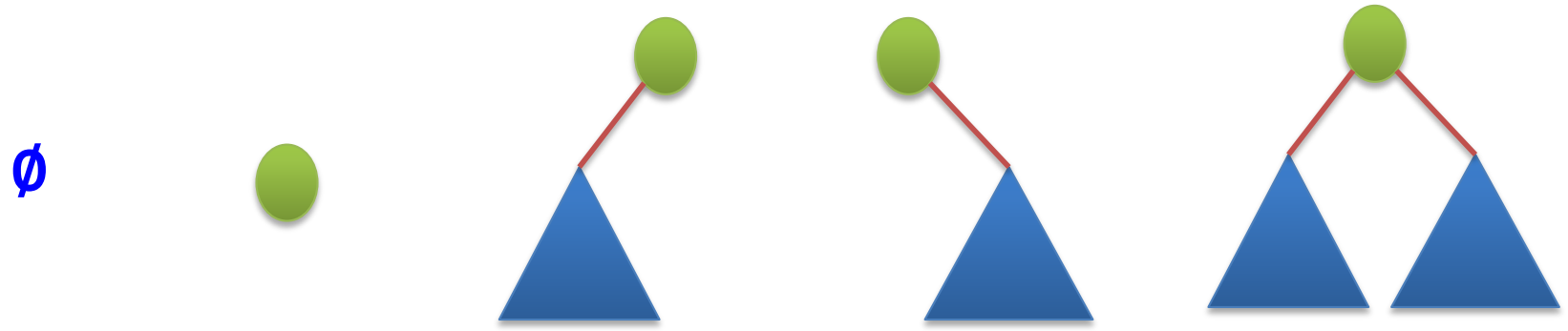
- 二叉树也称为二分树，它是有限的结点集合，这个集合或者是空，或者由一个根结点和两棵互不相交的称为左子树和右子树的二叉树组成。
- 二叉树中许多概念与树中的概念相同。
- 在含$n$个结点的二叉树中，所有结点的度小于等于2，通常用$n_0$表示叶子结点个数，$n_1$表示单分支结点个数，$n_2$表示双分支结点个数。



二叉树与度为2的树是不同的。

- 度为2的树至少有3个结点，而二叉树的结点数可以为0。
- 度为2的树不区分子树的次序，而二叉树中的每个结点最多有两个孩子结点，且必须要区分左右子树，即使在结点只有一棵子树的情况下也要明确指出该子树是左子树还是右子树。

归纳起来，二叉树的5种形态：



(a) 空二叉树

(b) 只有一个根结点的二叉树

(c) 右子树为空的二叉树

(d) 左子树为空的二叉树

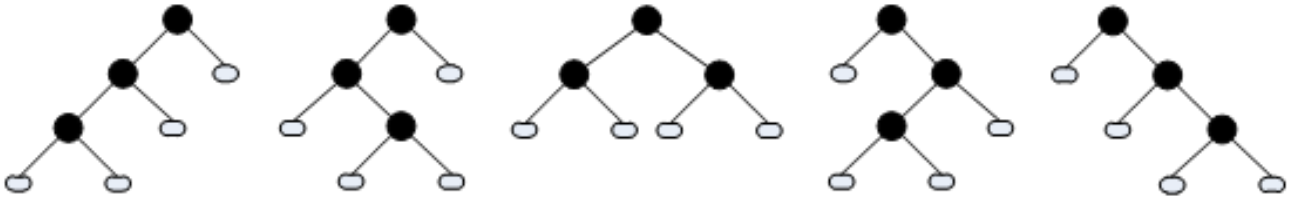
(e) 左、右子树非空的二叉树

Catalan数

## 二叉树问题

n个结点构成的二叉树，共有多少种情况？

例如，有3个结点（图中的黑点）的二叉树，可以构成5种二叉树：



解：

$$C_n = C_0C_{n-1} + C_1C_{n-2} + \ldots + C_{n-2}C_1 + C_{n-1}C_0 = \sum C_kC_{n-k}, \quad C_0 = 1$$

$C_0C_{n-1}$：右子树有0个结点 + 左子树有n-1个结点；

$C_1C_{n-2}$：右子树有1个结点 + 左子树有n-2个结点；

……

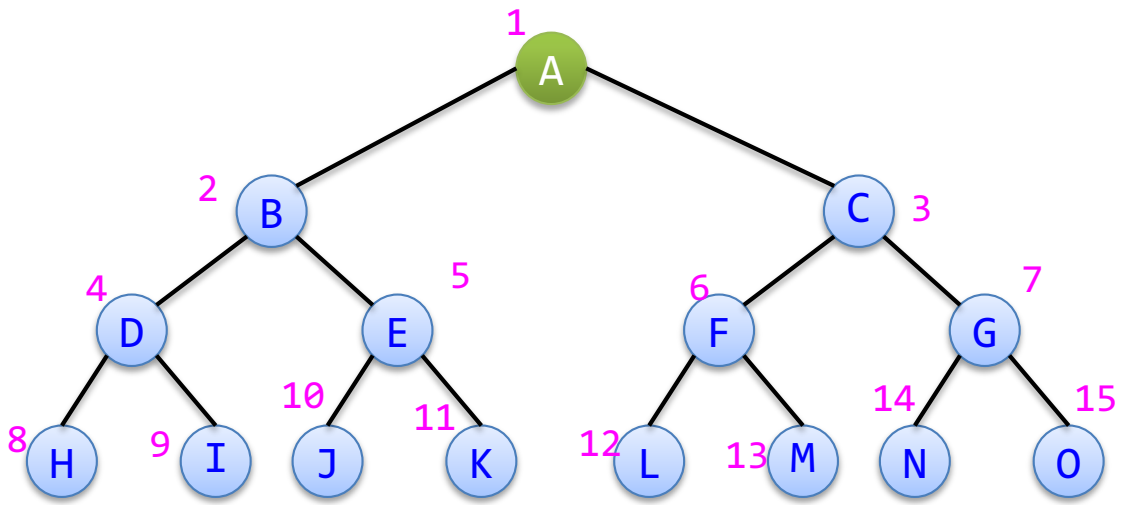
$C_{n-1}C_0$：右子树有n-1个结点 + 左子树有0个结点；

## 2.二叉树抽象数据类型的描述

ADT BTree
{
数据对象：
D={$a_i$ | 0≤i≤n-1，n≥0}  //为了简单，除特别外假设结点值为char
数据关系：
R={r}
r={<$a_i$，$a_j$> | $a_i$，$a_j$∈D，  0≤i，j≤n-1，当n=0时，称为空二叉树；
否则其中有一个根结点，其他结点构成根结点的互不相交的左、右子树，该左、右两棵子树也是二叉树 }
基本运算：
CreateBTree (str)：由二叉树括号表示串str创建二叉链。
DispBTree ()：返回二叉树的括号表示串。
FindNode(x)：在二叉树中查找值为x的结点。
int Height()：求二叉树的高度。
DestroyBTree(b)：销毁二叉树b。
…
}

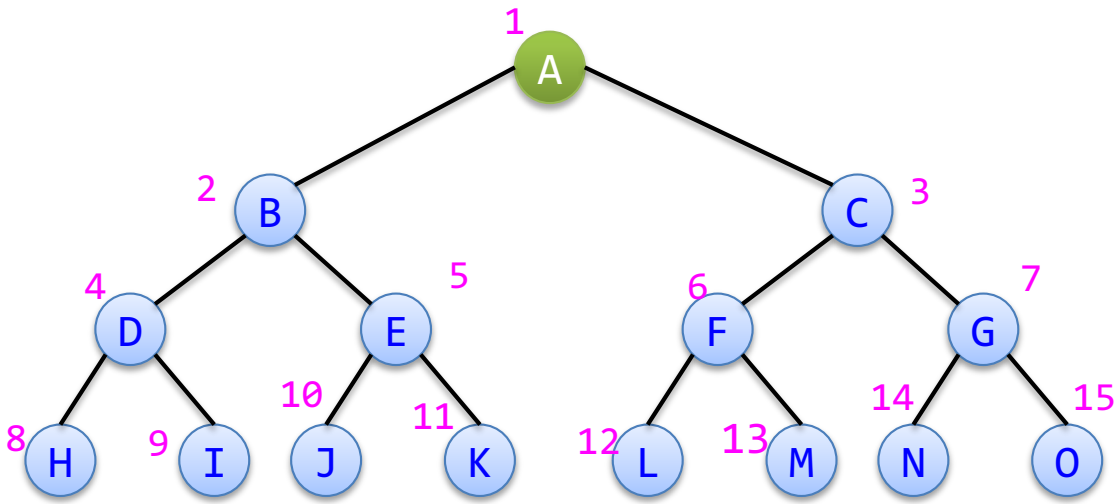
## 3．满二叉树和完全二叉树

- 在一棵二叉树中，如果所有分支结点都有左孩子结点和右孩子结点，并且叶子结点都集中在二叉树的最下一层，这样的二叉树称为满二叉树。
- 可以对满二叉树的结点进行层序编号，约定编号从树根为1开始，按照层数从小到大、同一层从左到右的次序进行。
- 满二叉树也可以从结点个数和树高度之间的关系来定义，即一棵高度为$h$且有$2^h-1$个结点的二叉树称为满二叉树。



满二叉树的特点如下：

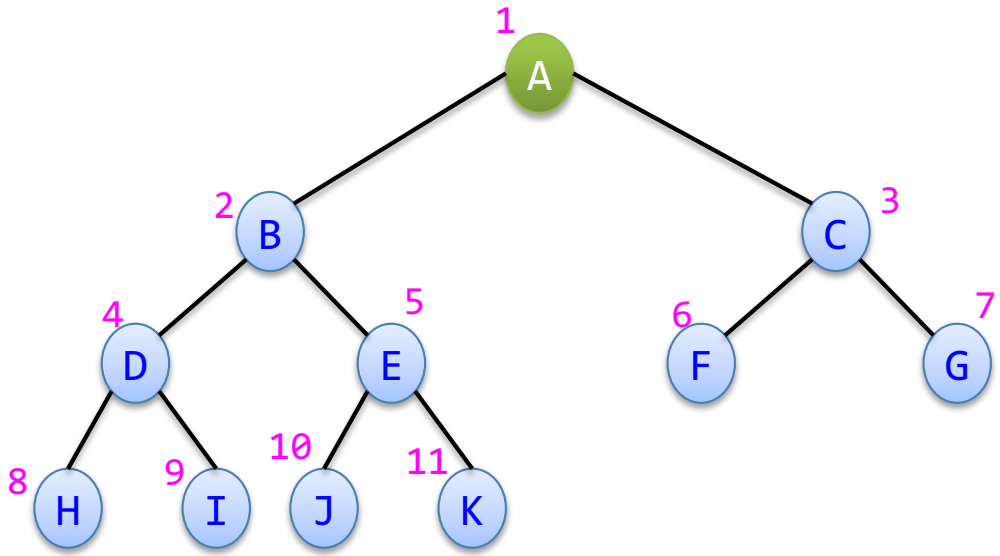
- 叶子结点都在最下一层。
- 只有度为0和度为2的结点。
- 含$n$个结点的满二叉树的高度为$\log_2(n+1)$，叶子结点个数为$\lfloor n/2 \rfloor+1$，度为2的结点个数为$\lfloor n/2 \rfloor$。




$n$=15

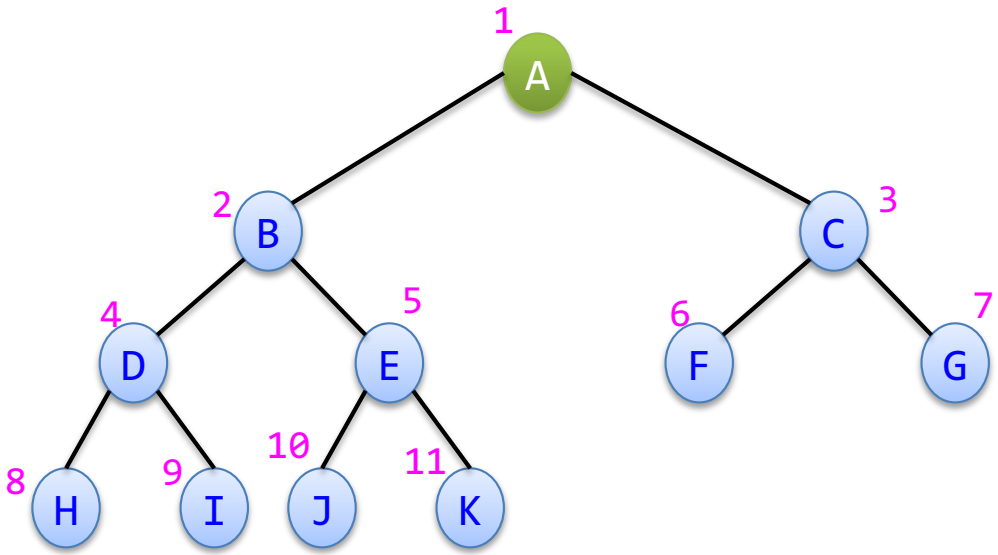
$h=\log_2(n+1)=4$

- 若二叉树中最多只有最下面两层的结点的度数可以小于2，并且最下面一层的叶子结点都依次排列在该层最左边的位置上，则这样的二叉树称为完全二叉树。
- 同样可以对完全二叉树中每个结点进行层序编号，编号的方法同满二叉树相同，图中每个结点外边的数字为对该结点的编号。



完全二叉树的特点如下：

- 叶子结点只可能出现在最下面两层中。
- 对于最大层次中的叶子结点，都依次排列在该层最左边的位置上。
- 如果有度为1的结点，只可能有一个，且该结点只有左孩子而无右孩子；
- 按层序编号后，一旦出现某结点（其编号为i）为叶子结点或只有左孩子，则编号大于i的结点均为叶子结点。



## 7.2.2 二叉树性质

**性质1** 非空二叉树上叶结点数等于双分支结点数加1。即$n_0=n_2+1$。

**证明：**

- 总结点数$n=n_0+n_1+n_2$。
- 一个度为1的结点贡献1个度，一个度为2的结点贡献2个度，所以总的度数$=n_1+2n_2$。
- 总的度数=总分支数$=n-1$。
- 则$n_1+2n_2=n_0+n_1+n_2-1$，求出$n_0=n_2+1$。

归纳

**在二叉树中计算结点时常用的关系式有：**
**①所有结点的度之和$=n-1$**
**②所有结点的度之和$=n_1+2n_2$**
**③$n=n_0+n_1+n_2$。**

**性质2** 非空二叉树上第$i$层上至多有$2^{i-1}$个结点，这里应有$i \geq 1$。

由树的性质2可推出。

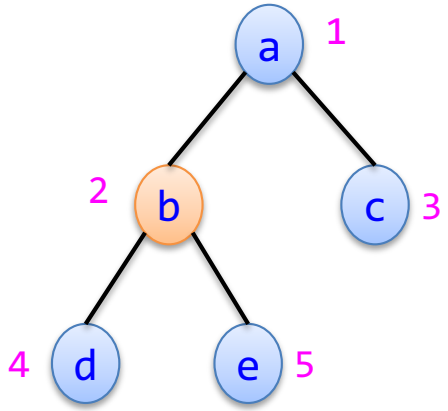
**性质3** 高度为$h$的二叉树至多有$2^h-1$个结点（$h \geq 1$）。

由树的性质3可推出。

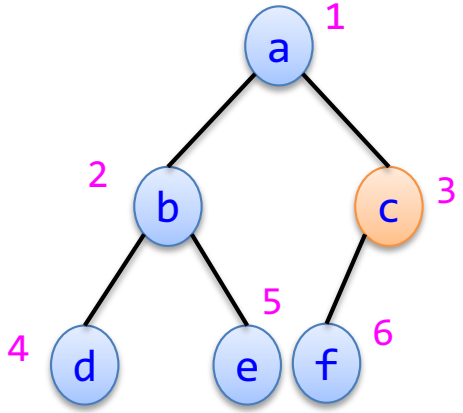
**性质4** 完全二叉树（结点个数为$n$）层序编号后的性质。

（1）若完全二叉树的根结点编号为1，对于编号为$i$（$1 \leqslant i \leqslant n$）的结点有：

- 若$i \leqslant \lfloor n/2 \rfloor$，即$2i \leqslant n$，则编号为$i$的结点为分支结点，否则为叶子结点，也就是说，最后一个分支结点的编号为$n/2$。
- 若$n$为奇数，则$n_1=0$，每个分支结点都是双分支结点；若$n$为偶数，则$n_1=1$，只有一个单分支结点。

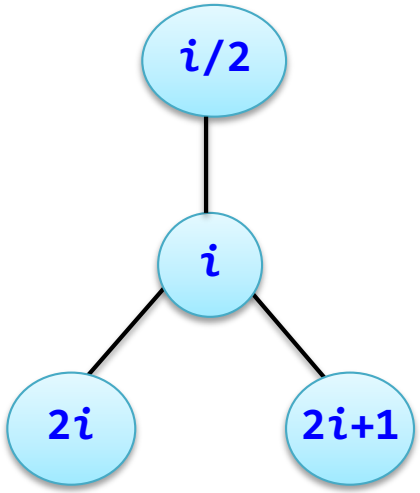



$n=5$ ⇨ $n_1=0$，最后分支结点为$n/2=2$



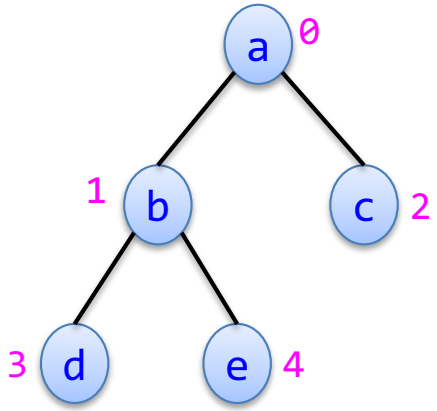

$n=6$ ⇨ $n_1=1$，最后分支结点为$n/2=3$

- 若编号为$i$的结点有左孩子结点，则左孩子结点的编号为$2i$；若编号为$i$的结点有右孩子结点，则右孩子结点的编号为$2i+1$。
- 若编号为$i$的结点有左兄弟结点，左兄弟结点的编号为$i-1$，若有右兄弟结点，右兄弟结点的编号为$i+1$。
- 若编号为$i$的结点有双亲结点，其双亲结点的编号为$\lfloor i/2 \rfloor$。

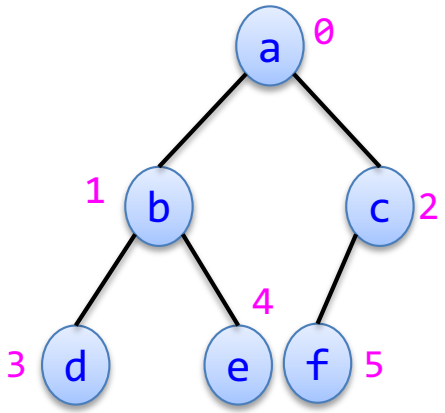


（2）若完全二叉树的根结点编号为0，对于编号为$i$（$0 \leqslant i \leqslant n-1$）的结点有：

- 若$i \leqslant \lfloor n/2 \rfloor - 1$，则编号为$i$的结点为分支结点，否则为叶子结点，也就是说，最后一个分支结点的编号为$\lfloor n/2 \rfloor - 1$。
- 若$n$为奇数，则$n_1=0$，每个分支结点都是双分支结点；若$n$为偶数，则$n_1=1$，只有一个单分支结点。

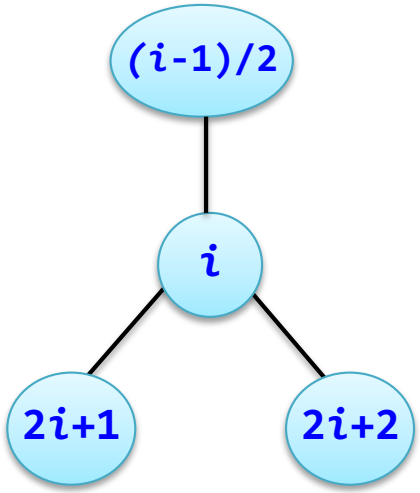



$n=5$ ⇨ $n_1=0$，最后分支结点为$n/2-1=1$




$n=6$ ⇨ $n_1=1$，最后分支结点为$n/2-1=2$

- 若编号为$i$的结点有左孩子结点，则左孩子结点的编号为$2i+1$；若编号为$i$的结点有右孩子结点，则右孩子结点的编号为$2i+2$。
- 若编号为$i$的结点有左兄弟结点，左兄弟结点的编号为$i-1$，若有右兄弟结点，右兄弟结点的编号为$i+1$。
- 若编号为$i$的结点有双亲结点，其双亲结点的编号为$\lfloor (i-1)/2 \rfloor$。



**性质5** 具有$n$个（$n>0$）结点的完全二叉树的高度为$\lceil \log_2(n+1) \rceil$或$\lfloor \log_2 n \rfloor+1$。

由完全二叉树的定义和树的性质4可推出。

**归纳**

- 一棵完全二叉树中，由结点总数$n$可以确定其树形。
- $n_1$只能是0或1，当$n$为偶数时，$n_1=1$，当$n$为奇数时，$n_1=0$。
- 层序编号（从1开始）为$i$的结点层次恰好为$\lceil \log_2(i+1) \rceil$或者$\lfloor \log_2 i \rfloor+1$。

【例7.5】一棵含有882个结点的二叉树中有365个叶子结点，求度为1的结点个数和度为2的结点个数。

这里$n=882$，$n_0=365$。

- 由二叉树的性质1可知$n_2=n_0-1=364$。
- $n=n_0+n_1+n_2$，即$n_1=n-n_0-n_2=882-365-364=153$。
- 所以该二叉树中度为1的结点和度为2的结点个数分别是153和364。

【例7.6】一棵完全二叉树中有501个叶子结点，则至少有多少个结点。

该二叉树中有，$n_0=501$。

- 由二叉树性质1可知$n_0=n_2+1$，所以$n_2=n_0-1=500$。
- $n=n_0+n_1+n_2=1001+n_1$，由于完全二叉树中$n_1=0$或$n_1=1$，则$n_1=0$时结点个数最少，此时$n=1001$，即至少有1001个结点。

【例7.7】含有60个叶子结点的二叉树的最小高度是多少？

- 在该二叉树中，$n_0=60$，$n_2=n_0-1=59$，$n=n_0+n_1+n_2=119+n_1$。
- 当$n_1=0$且为完全二叉树时高度最小。
- 此时高度$h=\lceil \log_2(n+1) \rceil = \lceil \log_2 120 \rceil =7$。

【例7.8】已知一棵非空满二叉树中有31个分支结点，则总结点个数是多少？

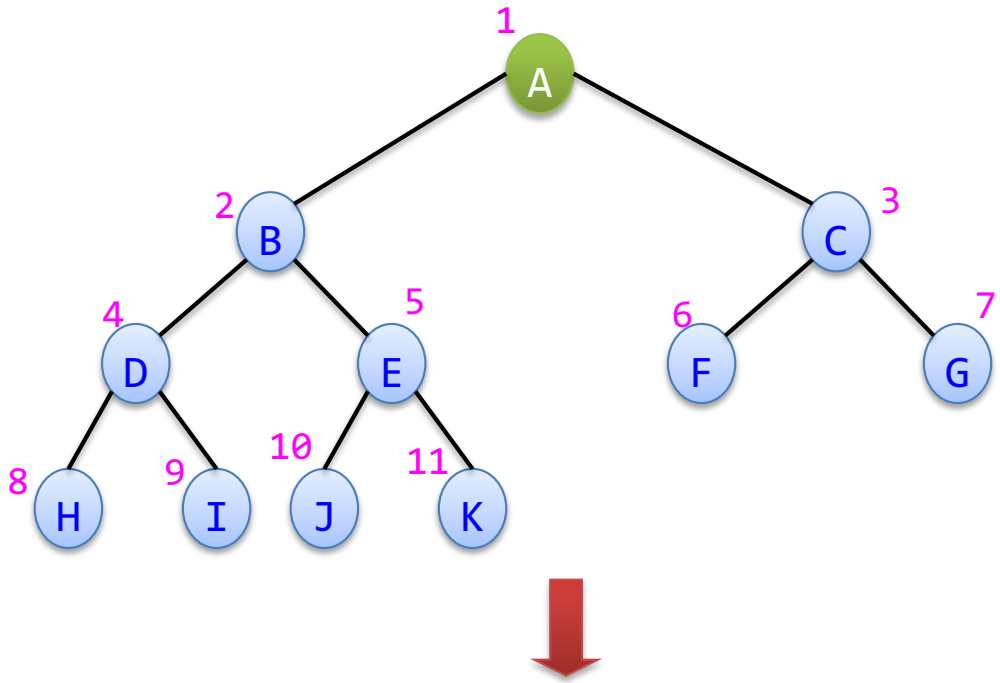
- $n_2+n_1=31$，而$n_1=0$，所以$n_2=31$（双分支结点个数）。
- $n_0=n_2+1=32$（二叉树性质），$n=n_0+n_1+n_2=63$。

# 7.2.3 二叉树存储结构

## 1. 二叉树的顺序存储结构

- 顺序存储一棵二叉树时，就是用一组连续的存储单元存放二叉树。
- 由二叉树的性质4可知，对于完全二叉树（或满二叉树），树中结点层序编号可以唯一地反映出结点之间的逻辑关系，所以可以用一维数组按从上到下、从左到右的顺序存储树中所有结点值，通过数组元素的下标关系反映完全二叉树或满二叉树中结点之间的逻辑关系。

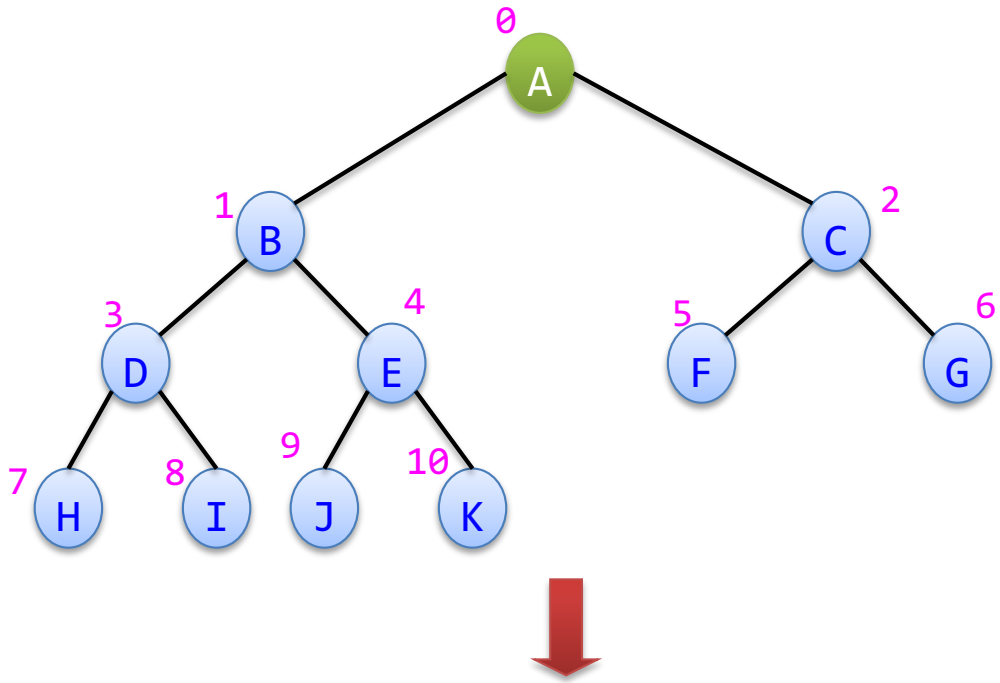
## 一棵完全二叉树的顺序存储结构




| 位置 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | … |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| sb | A | B | C | D | E | F | G | H | I | J | K | # | # | # | # |

sb的下标从1开始

或者

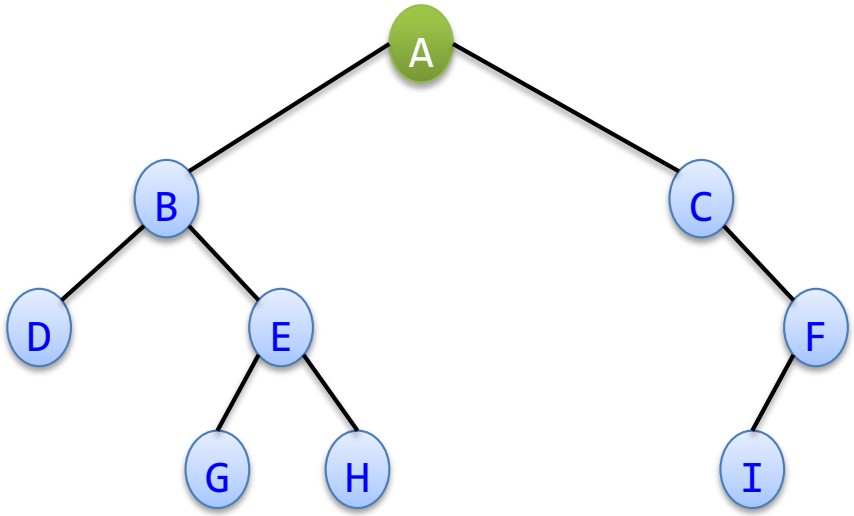


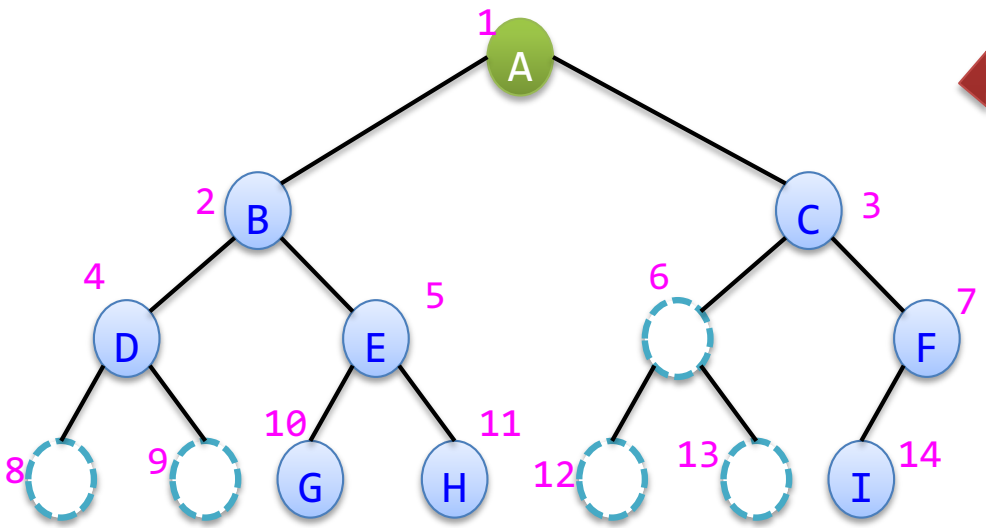

| 位置 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | … |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| sb | A | B | C | D | E | F | G | H | I | J | K | # | # | # | # |

sb的下标从0开始

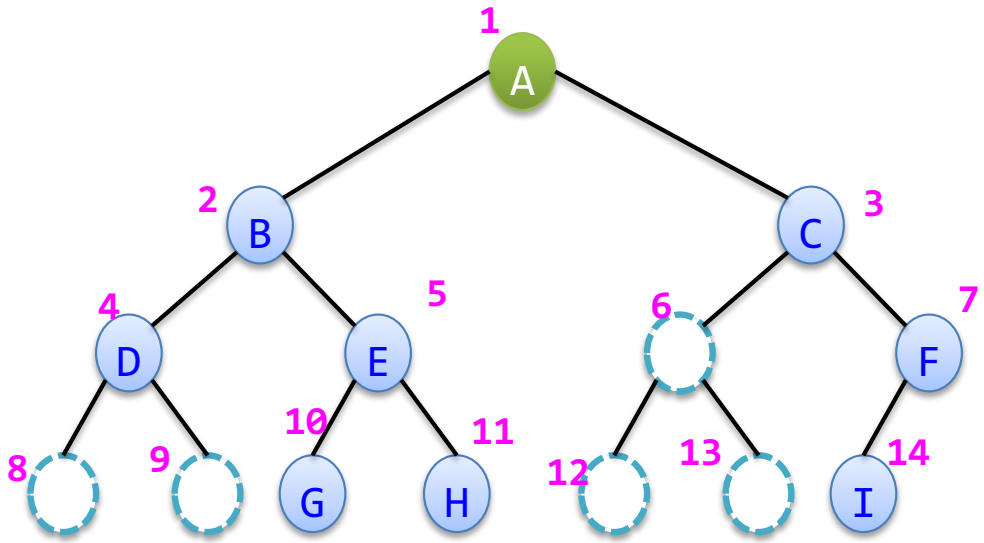
```
string sb="ABCDEFGHIJK####";
```

## 一般的二叉树的顺序存储结构




- 增添空结点补齐为一棵完全二叉树
- 并对所有结点进行编号

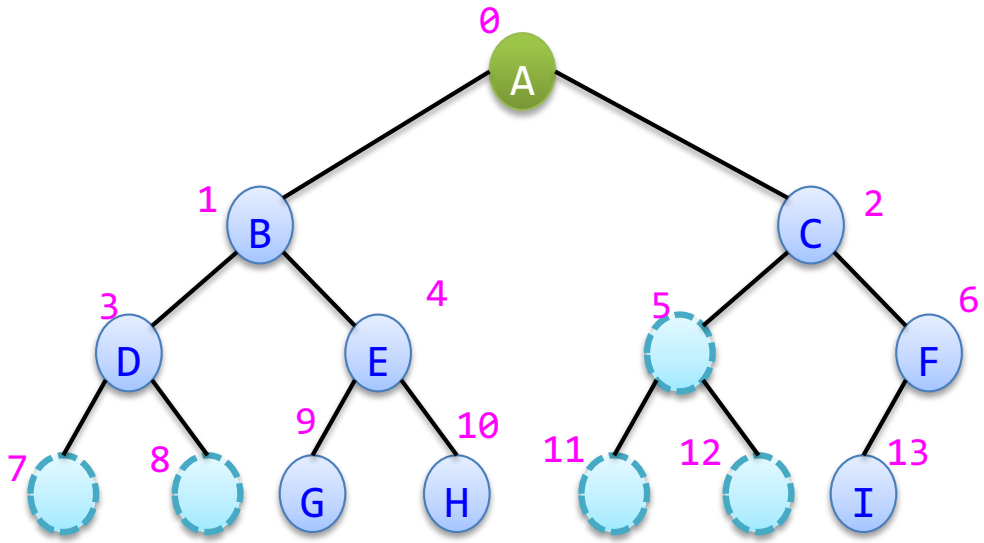



仅保留实际存在的结点值，其他为空

| 位置 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | … |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| sb | A | B | C | D | E | # | F | # | # | G | H | # | # | I | # |

sb的下标从1开始

或者




仅保留实际存在的结点值，其他为空

| 位置 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | … |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| sb | A | B | C | D | E | # | F | # | # | G | H | # | # | I | # |

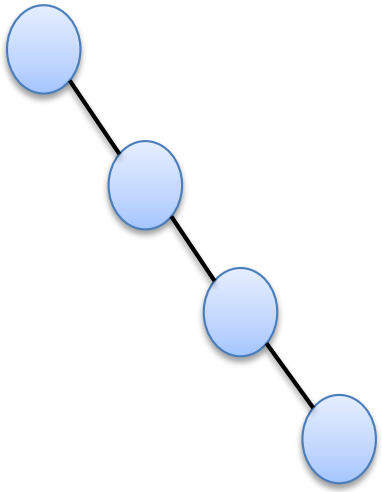
sb的下标从0开始

```
string sb="ABCDE#F##GH###I##";
```

- 二叉树顺序存储结构采用字符串（假设每个结点值为单个字符）或者数组存放。
- 当二叉树中某结点为空结点或无效结点（不存在该编号的结点）时，对应位置的值用特殊值（如'#'）表示。

## 优缺点

- 完全二叉树或满二叉树采用顺序存储结构比较合适
- 如果需要增加很多空结点才能将一棵二叉树改造成为一棵完全二叉树，采用顺序存储结构会造成空间的大量浪费，这时不宜用顺序存储结构。

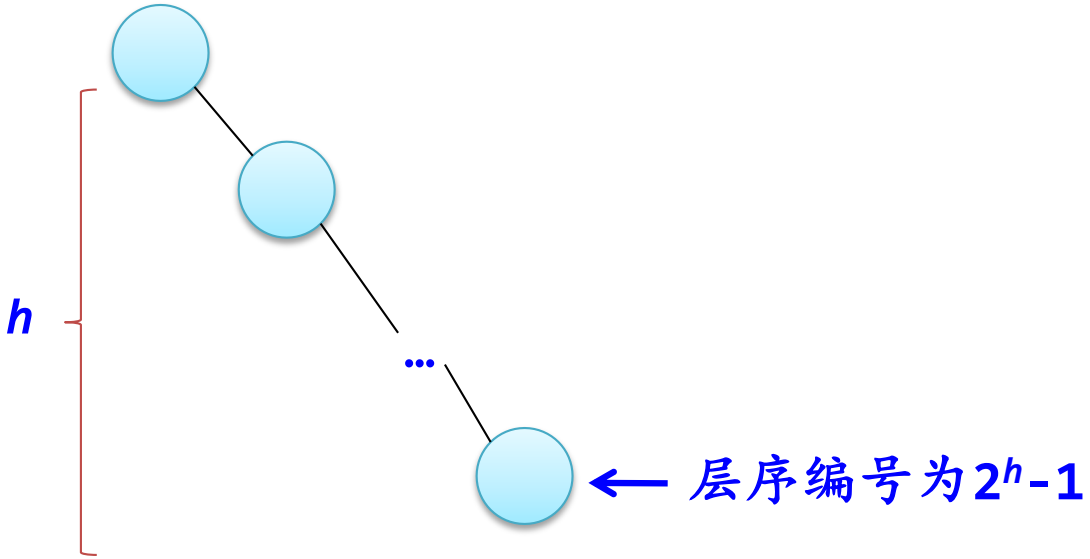


- $h=4$
- $MaxSize=2^4-1=15$
- 空间利用率=4/15=27%

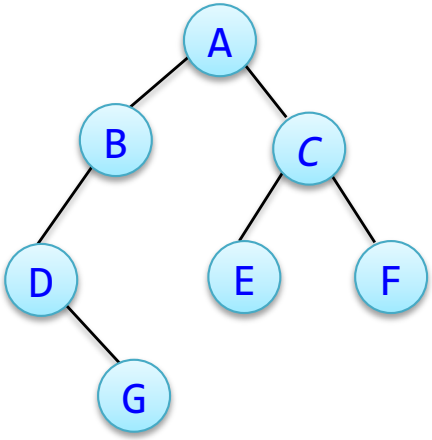
【例7.9】棵高度为$h$的并且只有$h$个结点的二叉树，采用顺序存储结构存放在$R[1..n]$中，则$n$应该至少是（ ）。

A.$2h$　　B.$2h-1$　　C.$2^h-1$　　D.$2^h$

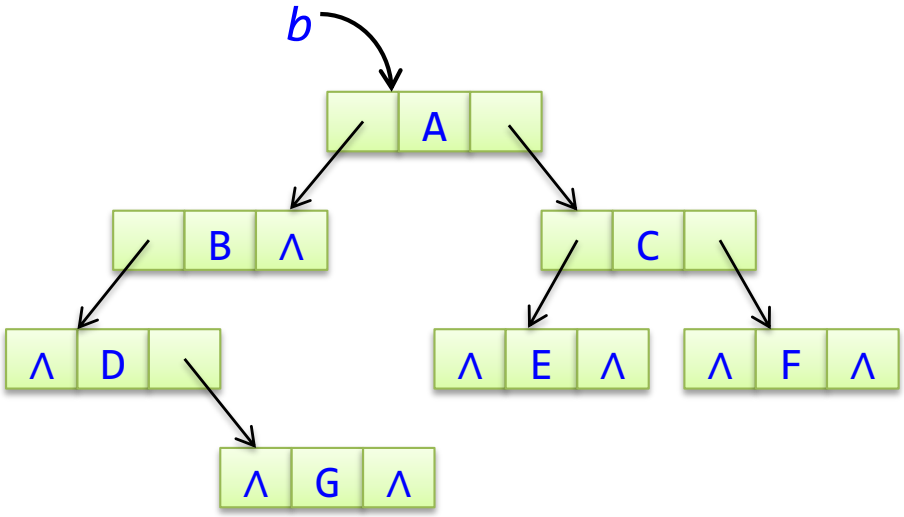
可能是一棵有斜树，最后一个结点的层序编号为$2^h-1$。C



# 2．二叉树的链式存储结构

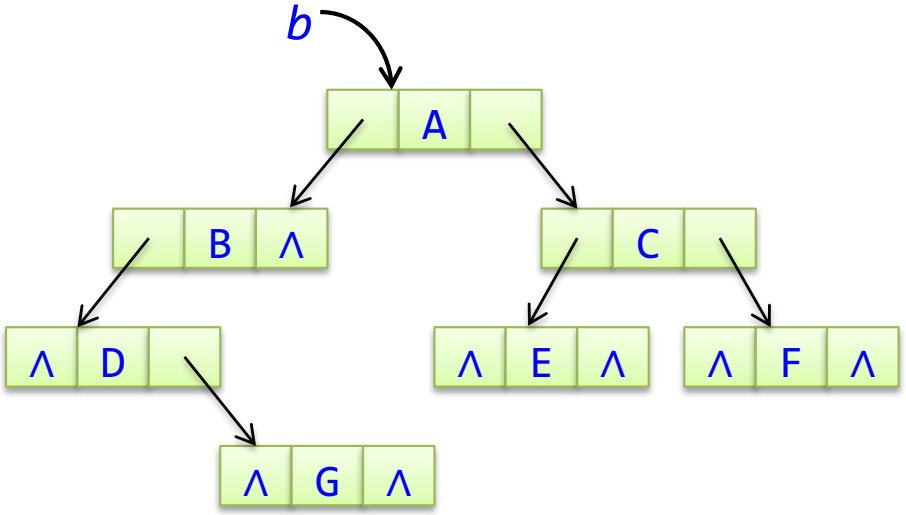



二叉链存储结构



## 对应的二叉链结点类型BTNode

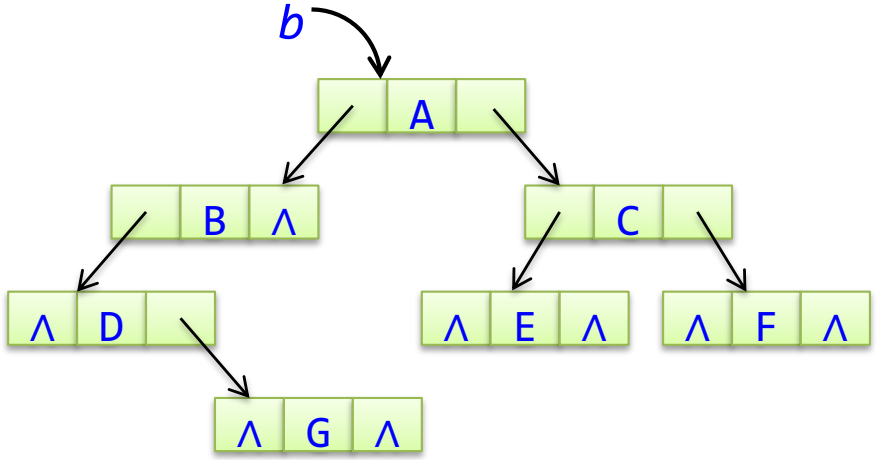
```
struct BTNode                                   //二叉链中结点类型
{   char data;                                  //数据元素
    BTNode* lchild;                             //指向左孩子结点
    BTNode* rchild;                             //指向右孩子结点

    BTNode():lchild(NULL),rchild(NULL) {}       //构造函数
    BTNode(char d)                              //重载构造函数
    {   data=d;
        lchild=rchild=NULL;
    }
};
```



## 优缺点

- 相对于顺序存储结构，二叉链方便二叉树的修改，对于普通二叉树和完全二叉树同样适合二叉链存储。
- 在二叉链中查找一个结点的孩子十分方便，但查找一个结点的双亲结点需要遍历二叉树。

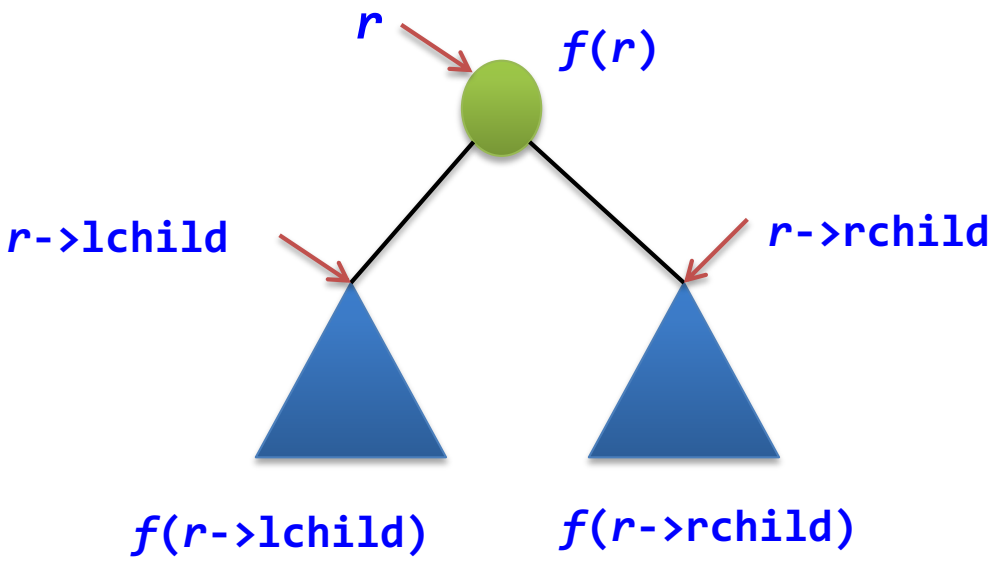
【例7.11】含有$n$个结点的二叉树采用二叉链存储结构，其中空指针域个数是多少？




- 每个结点2个指针域，共$2n$个指针域
- 除了根结点外，每个结点被一个非空指针所指向
- 共有$n-1$个非空指针域
- 空指针域的个数$=2n-(n-1)=n+1$。

# 7.2.4 二叉树的递归算法设计

一般地，二叉树的递归结构如下：




- 对于二叉树r，设f(r)是求解的“大问题”。
- f(r->lchild)和f(r->rchild)为“小问题”。
- 假设f(r->lchild)和f(r->rchild)是可求的，在此基础上得出f(r)和f(r->lchild)、f(r->rchild)之间的关系，从而得到递归体。
- 再考虑r=NULL或只有一个结点的特殊情况，从而得到递归出口。

例如，假设二叉树中所有结点值为整数，采用二叉链存储结构，求该二叉树r中所有结点值之和。

- 设f(r)为二叉树r中所有结点值之和。
- 则f(r->lchild)和f(r->rchild)分别求根结点r的左、右子树的所有结点值之和。
- 显然有f(r)=r->data+f(r->lchild)+f(r->rchild)。
- 当r=NULL时f(r)=0，从而得到以下递归模型：

$$f(r)=\begin{cases}0 & \text{当 } r=NULL\\ r->data+f(r->lchild)+f(r->rchild) & \text{其他情况}\end{cases}$$

```
int Sum(BTNode* r)                    //计算以b为根的二叉树的结点值之和
{  if (r==NULL) return 0;
   else return r->data+Sum(r->lchild)+Sum(r->rchild);
}
```

# 7.2.5 二叉树的基本运算及其实现

为了简单，本节讨论的二叉树中所有结点值为单个字符。逻辑结构采用括号表示串，存储结构采用二叉链。

## 1．二叉树类设计

```
class BTree                          //二叉树类
{   BTNode* r;                       //二叉树的根结点r
public:
    BTree()                          //构造函数,建立一棵空树
    {   r=NULL;   }
    //二叉树的基本运算
};
```

## 2.二叉树的基本运算算法实现

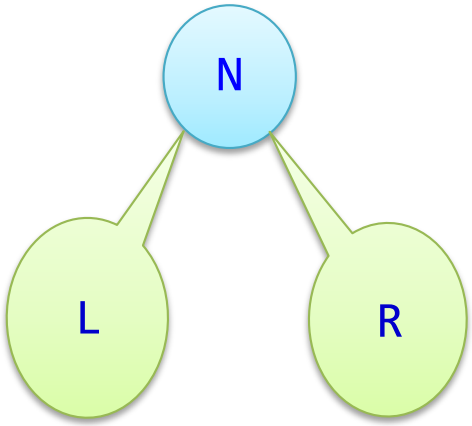
### (1) 创建二叉树：CreateBTree(str)

由正确的二叉树括号表示串 ⇨ 二叉链存储结构

正确的二叉树括号表示串中只有4类字符：

- 单个字符：结点的值
- (：表示一棵子树的开始
- )：表示一棵子树的结束
- ，：表示一棵右子树的开始

## 算法设计：




- 先构造根结点***N***，再构造左子树***L***，最后构造右子树***R***
- 构造右子树***R***时，找不到***N***了，所以需要保存***N***
- 而括号（子树）是按最近原则匹配的，所以使用一个栈保存***N***

用ch遍历采用括号表示法表示二叉树的字符串str:

*A*(*B*(*D*(, *G*)), *C*(*E*, *F*))

① 若ch='(': 则将前面刚创建的结点作为双亲结点进栈，并置flag=true，表示开始处理左孩子结点；

② 若ch=')': 表示栈顶结点的左、右孩子结点处理完毕，退栈；

③ 若ch=',': 表示开始处理右孩子结点，置flag=false；

④ 其他情况（结点值）：

创建*p*结点用于存放ch；

当flag=true时，将*p*结点作为栈顶结点的左孩子结点；

当flag=false时，将*p*结点作为栈顶结点的右孩子结点。

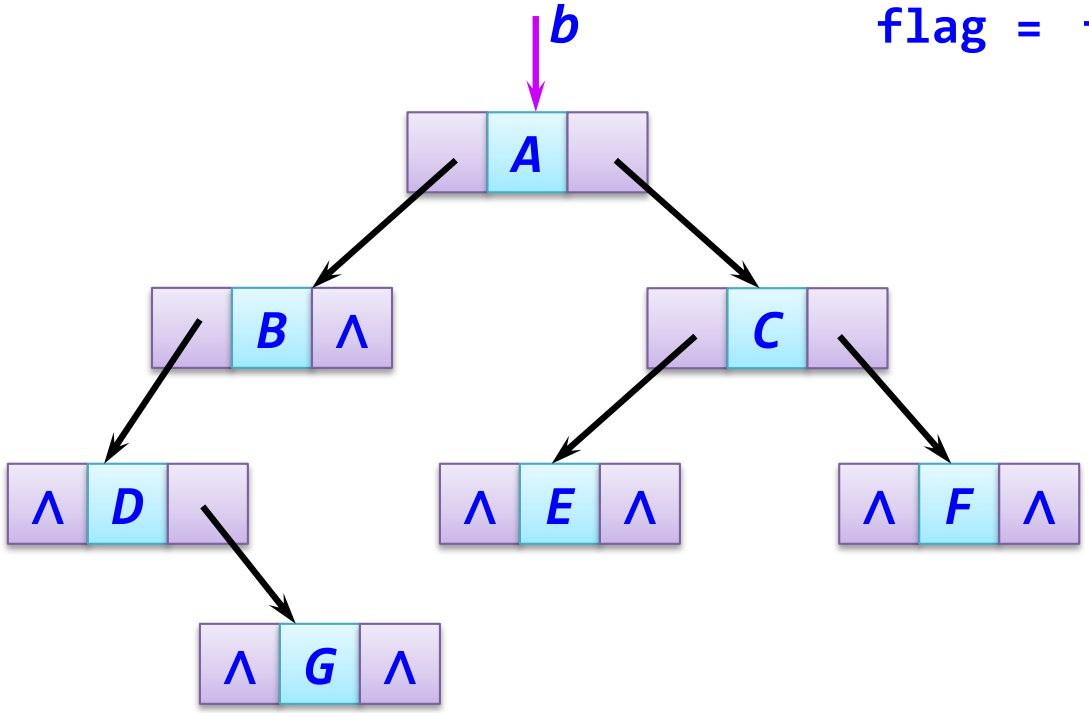
A ( B ( D ( , G ) ) , C ( E , F ) )

① 若ch='('：则将前面刚创建的结点作为双亲结点进栈，并置flag=true，表示开始处理左孩子结点；
② 若ch=')'：表示栈顶结点的左、右孩子结点处理完毕，退栈；
③ 若ch=','：表示开始处理右孩子结点，置flag=false；
④ 其他情况（结点值）：

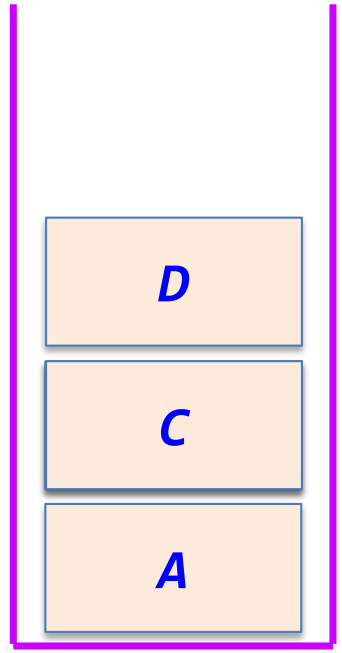
创建*p*结点用于存放ch；

当flag=true时，将*p*结点作为栈顶结点的左孩子结点；

当flag=false时，将*p*结点作为栈顶结点的右孩子结点。




flag = false



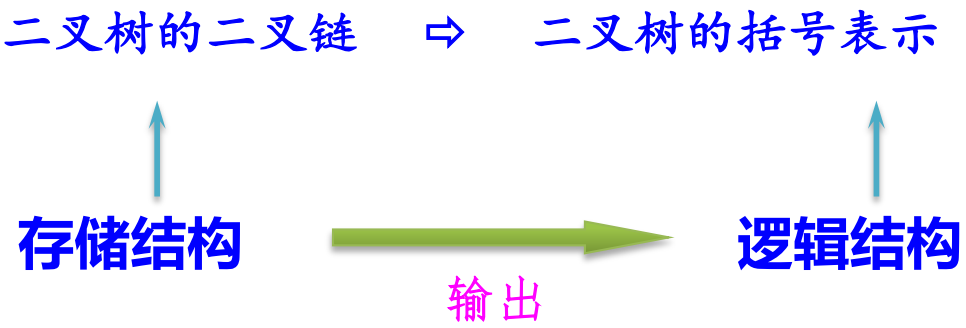

栈

二叉链创建完毕

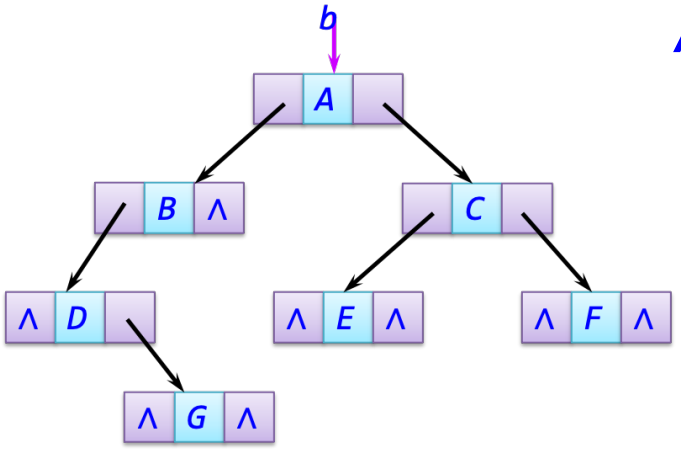
```
void CreateBTree(string str)          //创建以r为根结点的二叉链存储结构
{  stack<BTNode* > st;                //定义一个栈st
   BTNode* p;
   bool flag;
   int i=0;

   while (i<str.length())             //循环扫描str中每个字符
   {  switch(str[i])
      {
      case '(':
            st.push(p);               //刚刚新建的结点有孩子,将其进栈
            flag=true;
            break;
      case ')':
            st.pop();                 //栈顶结点的子树处理完，出栈
            break;
      case ',':
            flag=false;               //开始处理栈顶结点的右孩子
            break;
```

```
        default:
            p=new BTNode(str[i]);                  //新建一个结点p
            if (r==NULL)                           //若尚未建立根结点时
               r=p;                                //置结点p为根结点
            else                                   //已建立二叉树根结点
            {  if (flag && !st.empty())            //p作为栈顶结点的左孩子
                  st.top()->lchild=p;
               else if (!st.empty())               //p作为栈顶结点的右孩子
                  st.top()->rchild=p;
            }
            break;
        }
        i++;                                       //继续遍历
    }
}
```

## (2) 求二叉链的括号表示串DispBTree()




```
void DispBTree()                    //将二叉链转换成括号表示法
{
   DispBTree1(r);
}
```




A ( B ( D ( , G ) ) , C ( E , F ) )

```
void DispBTree1(BTNode* b)                    //被DispBTree函数调用
{  if (b!=NULL)
   {  cout << b->data;                        //输出根结点值
      if (b->lchild!=NULL || b->rchild!=NULL)
      {  cout << "(";                         //有孩子结点时输出"("
         DispBTree1(b->lchild);               //递归输出左子树
         if (b->rchild!=NULL)
            cout << ",";                      //有右孩子结点时输出","
         DispBTree1(b->rchild);               //递归输出右子树
         cout << ")";                         //输出")"
      }
   }
}
```

## （3）查找值为x的结点FindNode(x)

设*f*(b，*x*)在以b为根结点的二叉树中查找值为*x*的结点，找到后返回其地址，否则返回NULL。

| | |
|---|---|
| *f*(b，*x*)=NULL | 若*b*=NULL |
| *f*(b，*x*)=b | b->data=*x* |
| *f*(b，*x*)=*p*或者*q* | *p*=*f*(b->lchild,*x*)<br>*q*=f(b->rchild,x),p或者q非空 |
| *f*(b，*x*)=NULL | 其他情况 |

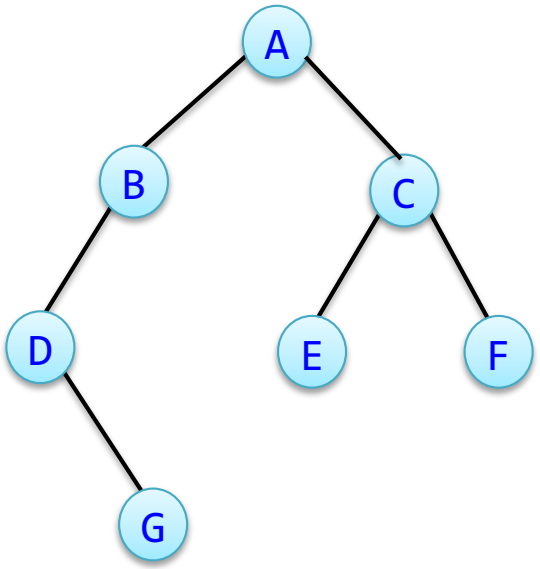
```
BTNode* FindNode(char x)                    //查找值为x的结点算法
{
   return FindNode1(r,x);
}

BTNode* FindNode1(BTNode* b,char x)  //被FindNode函数调用
{  BTNode* p,*q;
   if (b==NULL) return NULL;                //b为空时返回NULL
   else if (b->data==x) return b;           //b所指结点值为x时返回b
   else
   {  p=FindNode1(b->lchild,x);             //在左子树中查找
      q=FindNode1(b->rchild,x);             //在右子树中查找
      if (p!=NULL)
         return p;                          //在左子树中找到p结点，返回p
      else if (q!=NULL)                     //在右子树中找到q结点，返回q
         return q;
      else
         return NULL;                       //返回NULL
   }
}
```

合并操作

查找x='D'的结点

如何优化？？

- 先判断根结点，再分别遍历左、右子树。
- 若遍历左子树的结果不空，即已在左子树中找到该结点，仍需遍历右子树，是否有必要？




```
BTNode* FindNode1(BTNode* b,char x)
{  BTNode* p,*q;
   if (b==NULL) return NULL;
   else if (b->data==x) return b;
   else
   {  p=FindNode1(b->lchild,x);
      q=FindNode1(b->rchild,x);
      if (p!=NULL)
         return p;
      else if (q!=NULL)
         return q;
      else
         return NULL;
   }
}
```

合并操作

## 更优的解法

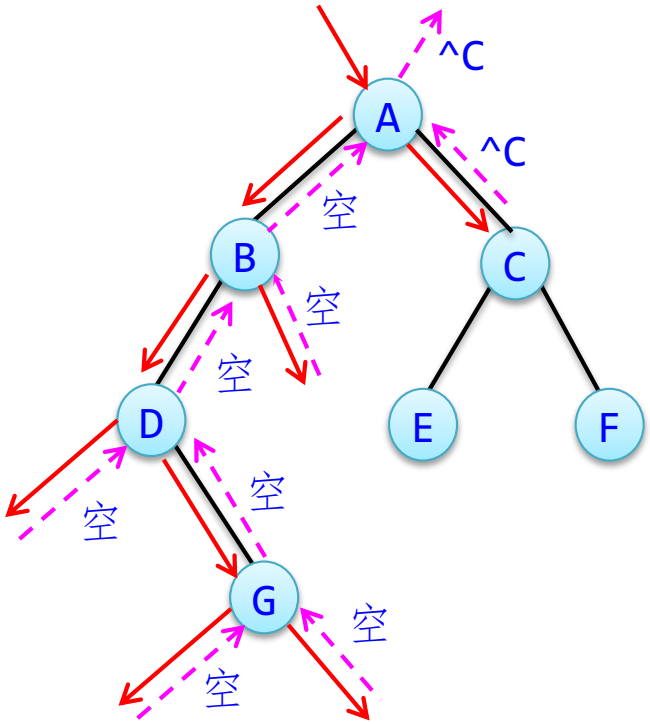
设*f*(b，*x*)在以b为根结点的二叉树中查找值为*x*的结点，找到后返回其地址，否则返回NULL。

| | |
|---|---|
| *f*(b，*x*)=NULL | 若*b*=NULL |
| *f*(b，*x*)=b | b->data=*x* |
| *f*(b，*x*)=*p* | 若在左子树中找到了，即<br>*p*=*f*(b->lchild，*x*)且p!=NULL |
| *f*(b，*x*)=*f*(b->rchild，*x*) | 其他情况 |

```
BTNode* FindNode(char x)                          //查找值为x的结点算法
{
   return FindNode1(r,x);
}

BTNode* FindNode1(BTNode* b,char x)               //被FindNode函数调用
{  BTNode* p;
   if (b==NULL) return NULL;                      //b为空时返回NULL
   else if (b->data==x) return b;                 //b所指结点值为x时返回b
   else
   {  p=FindNode1(b->lchild,x);                   //在左子树中查找
      if (p!=NULL)
         return p;                          //在左子树中找到p结点，返回p
      else
         return FindNode1(b->rchild,x);    //返回在右子树中查找结果
   }
}
```

## 查找x='C'的结点

- 先判断根结点，再遍历左子树，从左子树返回后可能遍历右子树。
- 返回时若左子树的结果不空则不遍历右子树，否则会遍历右子树。



## （4）求高度Height()

设以*b*为根结点二叉树的高度为*f*(*b*)，空树高度为0，非空树高度为左、右子树中较大的高度加1。

$$f(b) = 0 \qquad \text{若b=NULL}$$

$$f(b) = \text{MAX}\{f(\text{b->lchild}), f(\text{b->rchild})\}+1 \qquad \text{其他情况}$$

```
int Height()                                    //求二叉树高度的算法
{
   return Height1(r);
}

int Height1(BTNode* b)                          //被Height函数调用
{  if (b==NULL)                                 //空树的高度为0
      return 0;
   else
      return max(Height1(b->lchild),Height1(b->rchild))+1;
}
```

## （5）销毁二叉树：DestroyBTree(b)

设*f*(*b*)的功能是销毁以*b*为根结点二叉树，即释放其中所有结点的空间。对应的递归模型如下：

*f*(*b*) ≡ 不做任何事情　　当b=NULL

*f*(*b*) ≡ *f*(b->lchild); *f*(b->rchild); delete b;　　其他情况

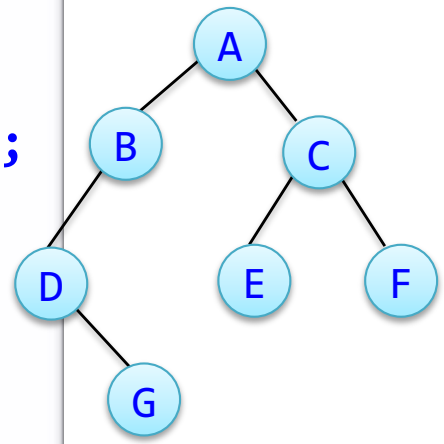
## （5）销毁二叉树：DestroyBTree(b)

```
void DestroyBTree(BTNode* b)          //释放所有的结点空间
{  if (b!=NULL)
   {  DestroyBTree(b->lchild);        //递归释放左子树
      DestroyBTree(b->rchild);        //递归释放右子树
      delete b;                       //释放根结点
   }
}
```

```
~BTree()                              //析构函数
{  DestroyBTree(r);                   //调用DestroyBTree()函数
   r=NULL;                            //置为空树
}
```

程序验证

```
#include"BTree.cpp"    //引用二叉树类BTree
int main()
{   string str="A(B(D(,G)),C(E,F))";
    char x='e';
    BTree bt;
    bt.CreateBTree(str);
    cout << "二叉树bt:"; bt.DispBTree(); cout << endl;
    cout << "bt的高度:" << bt.Height() << endl;
    if (bt.FindNode(x))
       cout << "bt中找到值为" << x << "的结点\n";
    else
       cout << "bt中没有找到值为" << x << "的结点\n";
    cout << "销毁二叉树\n";
    return 0;
}
```



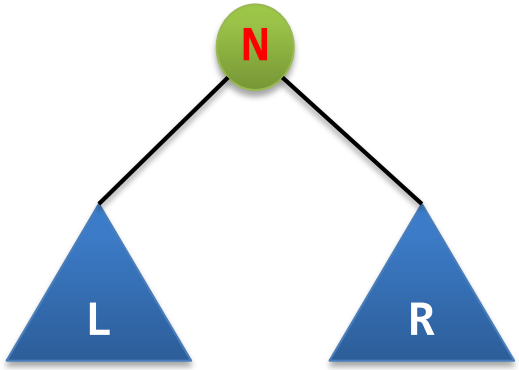
操作

所有代码存放在BTree.cpp中

# 7.3 二叉树先序、中序和后序遍历

## 7.3.1 二叉树遍历的概念

- 二叉树遍历是指按照一定次序访问二叉树中所有结点，并且每个结点仅被访问一次的过程。
- 设N为根结点，L、R分别为左、右子树，这6种遍历方法是NLR、LNR、LRN、NRL、RNL、RLN），若再规定先遍历左子树，后遍历右子树，则对于非空二叉树，可得到如下3种递归的遍历方法（即NLR、LNR和LRN）。

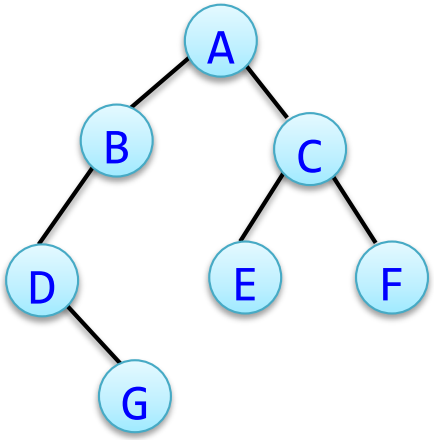


## 1）先序遍历

① 访问根结点。

② 先序遍历左子树。

③ 先序遍历右子树。

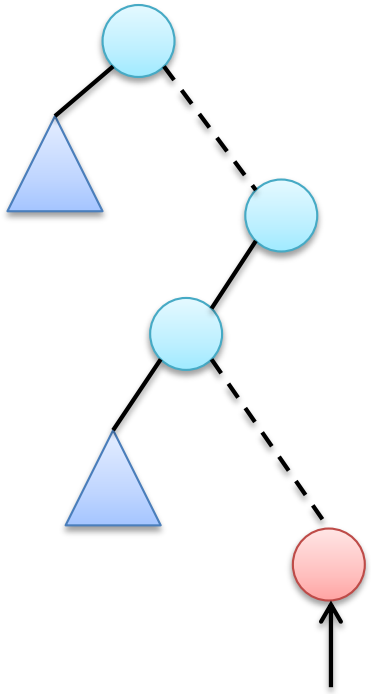



先序序列为：**ABDGCEF**

**说明**

在一棵二叉树的先序序列中，第一个元素即为根结点对应的结点值。

【例7.8】给出求一棵非空二叉树先序序列尾结点的过程。



没有左右孩子

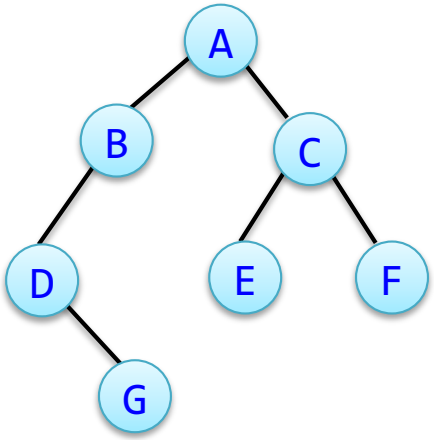
- p从根结点开始。
- 当前结点p没有左右孩子，返回p。
- 当前结点p有右子树 ⇨ 转向右孩子，
  否则 ⇨ 转向左孩子。

## 2）中序遍历

① 中序遍历左子树。

② 访问根结点。

③ 中序遍历右子树。




中序序列为：**DGBAECF**。

**说明**

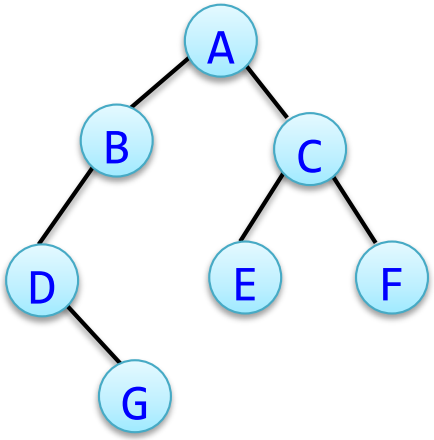
在一棵二叉树的中序序列中，根结点值将其序列分为前后两部分，前部分为左子树的中序序列，后部分为右子树的中序序列。

## 3）后序遍历

① 后序遍历左子树。

② 后序遍历右子树。

③ 访问根结点。




后序序列为：**GDBEFCA**。

**说明**

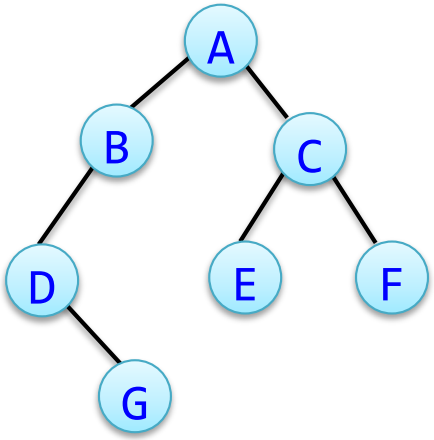
在一棵二叉树的后序序列中，最后一个元素即为根结点对应的结点值。

## 3）后序遍历

① 后序遍历左子树。

② 后序遍历右子树。

③ 访问根结点。




后序序列为：**GDBEFCA**。

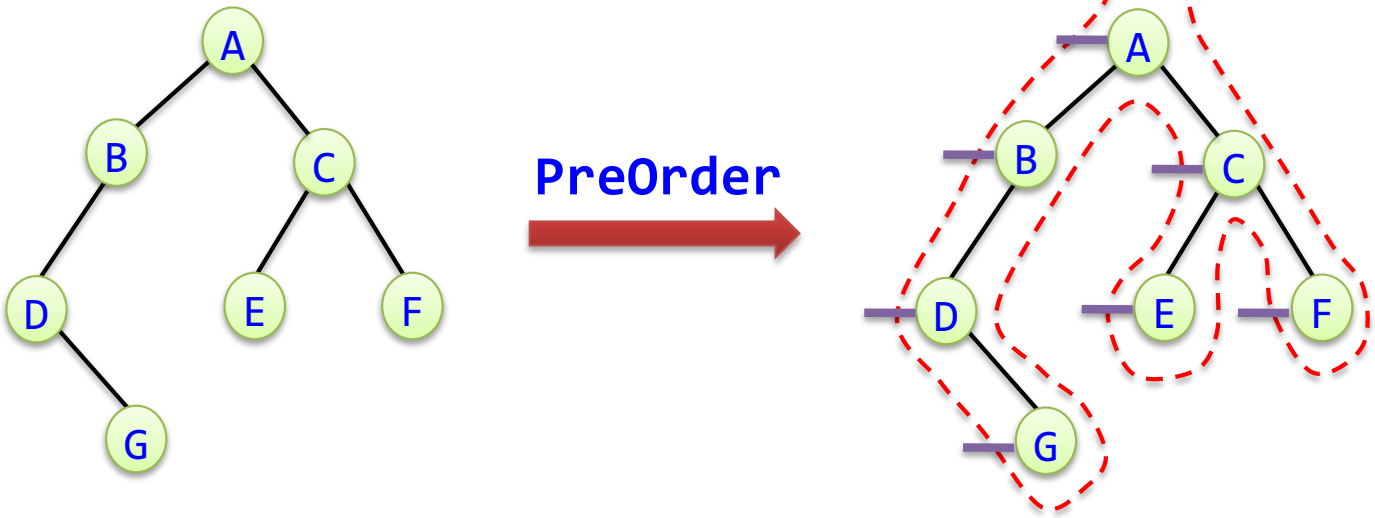
**说明**

在一棵二叉树的后序序列中，最后一个元素即为根结点对应的结点值。

# 7.3.2 先序、中序和后序遍历递归算法

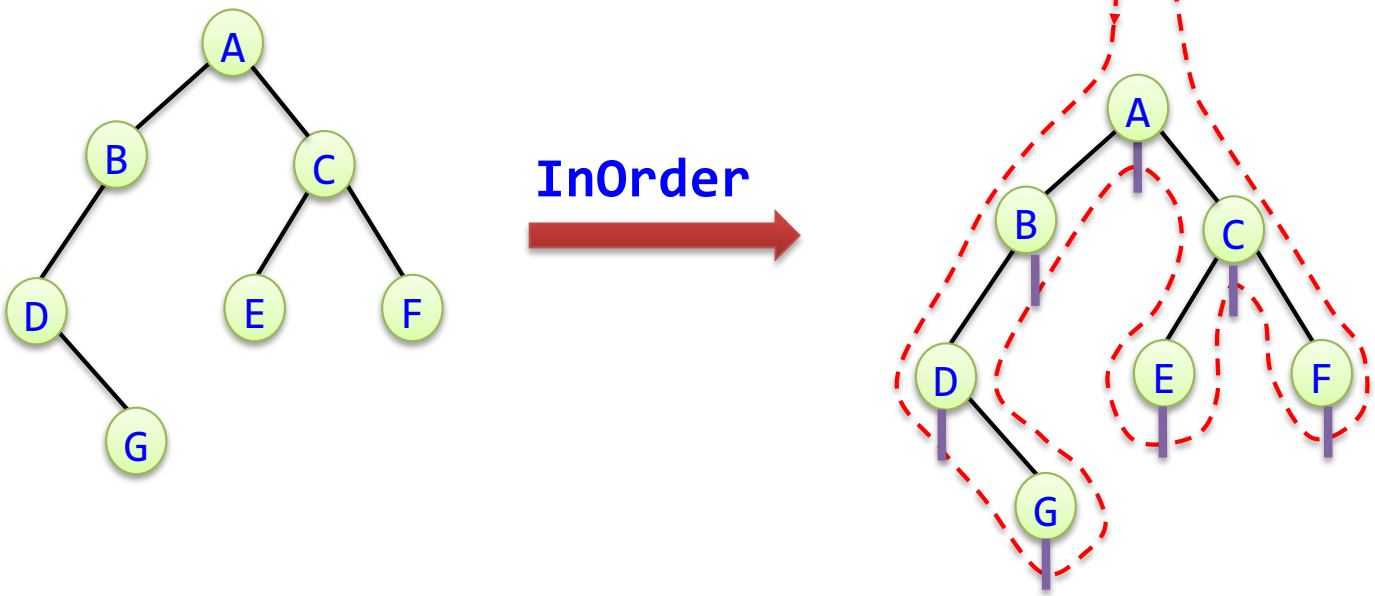
## 1）先序遍历的递归算法

```
void PreOrder11(BTNode* b)                  //被PreOrder函数调用
{  if (b!=NULL)
   {  cout << b->data;                      //访问根结点
      PreOrder11(b->lchild);                //先序遍历左子树
      PreOrder11(b->rchild);                //先序遍历右子树
   }
}

void PreOrder1(BTree& bt)                   //先序遍历的递归算法
{
   PreOrder11(bt.r);
}
```


A
B
C
D
E
F
G
PreOrder
A
B
C
D
E
F
G

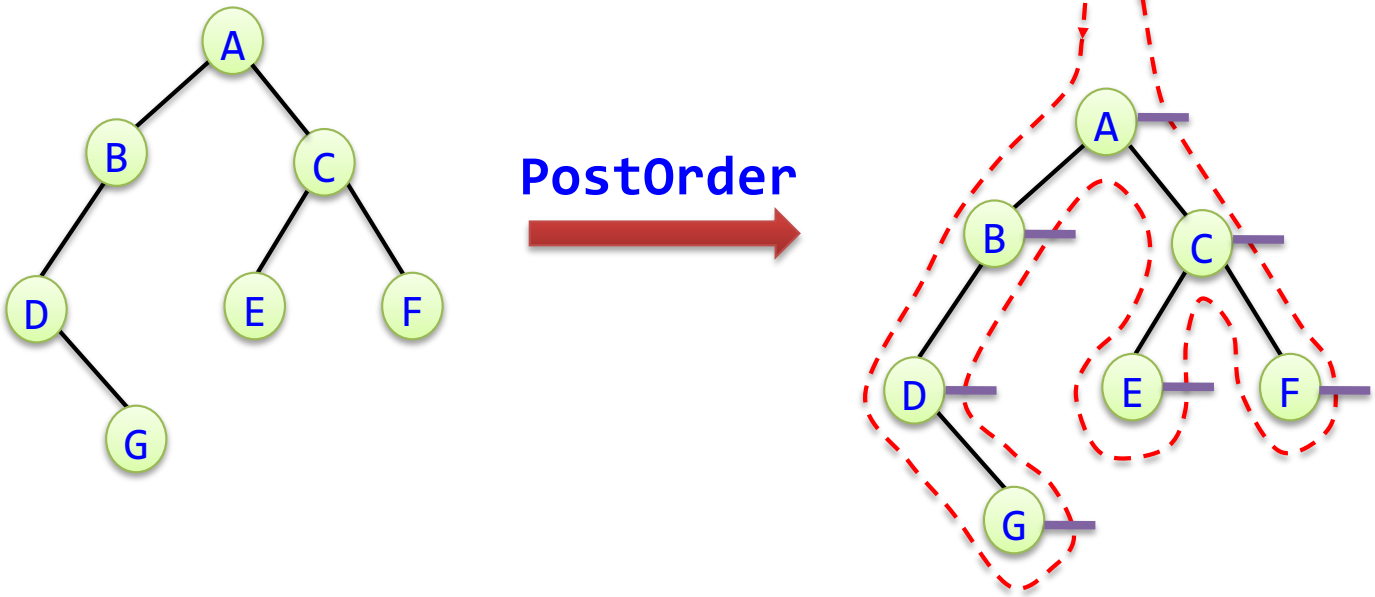
## 2）中序遍历的递归算法

```
void InOrder11(BTNode* b)                    //被InOrder函数调用
{  if (b!=NULL)
   {  InOrder11(b->lchild);                  //中序遍历左子树
      cout << b->data;                       //访问根结点
      InOrder11(b->rchild);                  //中序遍历右子树
   }
}

void InOrder1(BTree& bt)                     //中序遍历的递归算法
{
   InOrder11(bt.r);
}
```


A
B
C
D
E
F
G
InOrder
A
B
C
D
E
F
G

## 3）后序遍历的递归算法

```
void PostOrder11(BTNode* b)              //被PostOrder函数调用
{  if (b!=NULL)
   {  PostOrder11(b->lchild);            //后序遍历左子树
      PostOrder11(b->rchild);            //后序遍历右子树
      cout << b->data;                   //访问根结点
   }
}

void PostOrder1(BTree& bt)               //后序遍历的递归算法
{
   PostOrder11(bt.r);
}
```


A
B
C
D
E
F
G
PostOrder
A
B
C
D
E
F
G

## 7.3.3 递归遍历算法的应用

【例7.9】假设二叉树采用二叉链存储结构存储，设计一个算法求一棵给定二叉树中的结点个数。

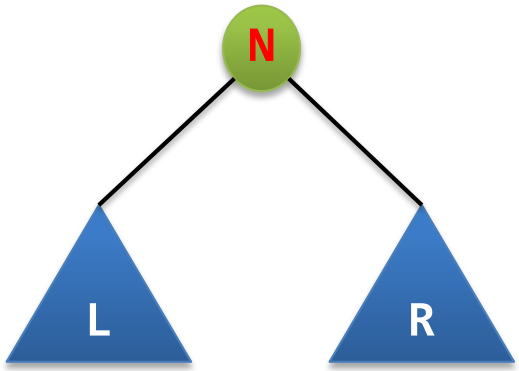
求一棵二叉树中的结点个数是以遍历算法为基础的，任何一种遍历算法都可以出一棵二叉树中的结点个数。

```
int NodeCount11(BTNode* b)
{  int m,n,k;
   if (b!=NULL)
   {  k=1;                              //根结点计数1
      m=NodeCount11(b->lchild);         //遍历求左子树的结点个数
      n=NodeCount11(b->rchild);         //遍历求右子树的结点个数
      return k+m+n;
   }
   else return 0;                       //空树结点个数为0
}

int NodeCount1(BTree& bt)               //基于先序遍历求结点个数
{
   return NodeCount11(bt.r);
}
```

```
int NodeCount21(BTNode* b)
{  int m,n,k;
   if (b!=NULL)
   {  m=NodeCount21(b->lchild);          //遍历求左子树的结点个数
      k=1;                               //根结点计数1
      n=NodeCount21(b->rchild);          //遍历求右子树的结点个数
      return k+m+n;
   }
   else return 0;                        //空树结点个数为0
}

int NodeCount2(BTree& bt)                //基于中序遍历求结点个数
{
   return NodeCount21(bt.r);
}
```

```
int NodeCount31(BTNode* b)
{   int m,n,k;
    if (b!=NULL)
    {   m=NodeCount31(b->lchild);          //遍历求左子树的结点个数
        n=NodeCount31(b->rchild);          //遍历求右子树的结点个数
        k=1;                               //根结点计数1
        return k+m+n;
    }
    else return 0;                         //空树结点个数为0
}

int NodeCount3(BTree& bt)                  //基于后序遍历求结点个数
{
    return NodeCount31(bt.r);
}
```

也可以从递归算法设计的角度来求解。设*f*(*b*)求二叉树*b*中所有结点个数，它是“大问题”，*f*(b->lchild)和*f*(b->rchild)分别求左、右子树的结点个数。




| | |
|---|---|
| *f*(*b*)=0 | 当*b*=NULL |
| *f*(*b*)=*f*(b->lchild)+*f*(b->rchild)+1 | 其他情况 |

$f(b)=0$ 当$b$=NULL

$f(b)=f(b->lchild)+f(b->rchild)+1$ 其他情况

```
int NodeCount41(BTNode* b)
{  if (b==NULL)
     return 0;                    //空树结点个数为0
   else
     return(NodeCount41(b->lchild)+NodeCount41(b->rchild)+1);
}

int NodeCount4(BTree& bt)     //基于递归设计方法求结点个数
{
   return NodeCount41(bt.r);
}
```

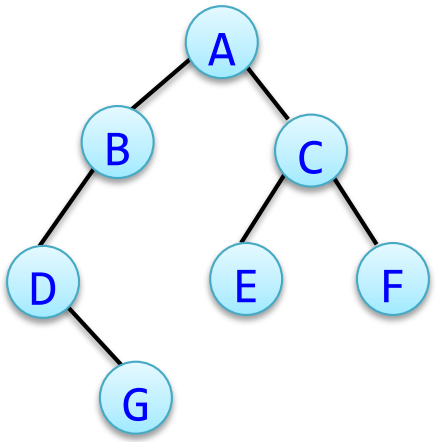
$f(b)=0$ 当$b$=NULL

$f(b)=f(b->lchild)+f(b->rchild)+1$ 其他情况

其中“+1”相当于访问结点，放在不同位置体现不同的递归遍历思路，NodeCount41()方法是将“+1”放在最后，体现出后序遍历的算法思路。

基于递归遍历思路和直接采用递归算法设计方法完全相同。实际上，当求解问题较复杂时，直接采用递归算法设计方法更加简单方便。

【例7.10】假设二叉树采用二叉链存储结构存储，设计一个算法按从左到右输出一棵二叉树中所有叶子结点值。




先序序列为：ABDGCEF

叶子？

中序序列为：DGBAECF

后序序列为：GDBEFCA

① 访问根结点。
② 先序遍历左子树。
③ 先序遍历右子树。

① 中序遍历左子树。
② 访问根结点。
③ 中序遍历右子树。

① 后序遍历左子树。
② 后序遍历右子树。
③ 访问根结点。

【例7.10】假设二叉树采用二叉链存储结构存储，设计一个算法按从左到右输出一棵二叉树中所有叶子结点值。

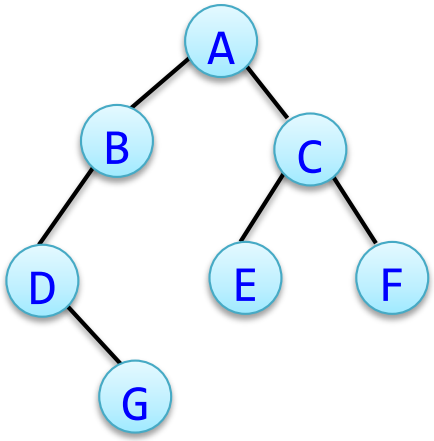
由于先序、中序和后序递归遍历算法都是按从左到右的顺序访问叶子结点的，所以本题可以基于这三种递归遍历算法求解。

## 先序

```
void PreOrder11(BTNode* b)                  //被PreOrder函数调用
{  if (b!=NULL)
   {  cout << b->data;                      //访问根结点

      PreOrder11(b->lchild);                //先序遍历左子树
      PreOrder11(b->rchild);                //先序遍历右子树
   }
}

void PreOrder1(BTree& bt)                   //先序遍历的递归算法
{
   PreOrder11(bt.r);
}
```

## 先序

```
void DispLeaf11(BTNode* b)
{  if (b!=NULL)
   {  if (b->lchild==NULL && b->rchild==NULL)
         cout << b->data << " ";      //根结点为叶子结点时输出
      DispLeaf11(b->lchild);          //输出左子树的叶子结点
      DispLeaf11(b->rchild);          //输出右子树的叶子结点
   }
}

void DispLeaf1(BTree& bt)
{
   DispLeaf11(bt.r);
}
```




遍历过程？

## 中序

```
void DispLeaf21(BTNode* b)
{  if (b!=NULL)
   {  DispLeaf21(b->lchild);          //输出左子树的叶子结点
      if (b->lchild==NULL && b->rchild==NULL)
         cout << b->data << " ";      //根结点为叶子结点时输出
      DispLeaf21(b->rchild);          //输出右子树的叶子结点
   }
}

void DispLeaf2(BTree& bt)
{
   DispLeaf21(bt.r);
}
```

## 后序

```
void DispLeaf31(BTNode* b)
{  if (b!=NULL)
   {  DispLeaf31(b->lchild);          //输出左子树的叶子结点
      DispLeaf31(b->rchild);          //输出右子树的叶子结点
      if (b->lchild==NULL && b->rchild==NULL)
         cout << b->data << " ";      //根结点为叶子结点时输出
   }
}

void DispLeaf3(BTree& bt)
{
   DispLeaf31(bt.r);
}
```

- 也可以直接采用递归算法设计方法求解。
- 设*f*(*b*)的功能是从左到右输出以*b*为根结点的二叉树的所有叶子结点值，为“大问题”，显然*f*(*b*->lchild)和*f*(*b*->rchild)是两个“小问题”。
- 当*b*不是叶子结点时，先调用*f*(*b*->lchild)再调用*f*(*b*->rchild)。
- 对应的递归模型*f*(*b*)如下：

*f*(*b*) ≡ 不做任何事件　　若*b*=NULL

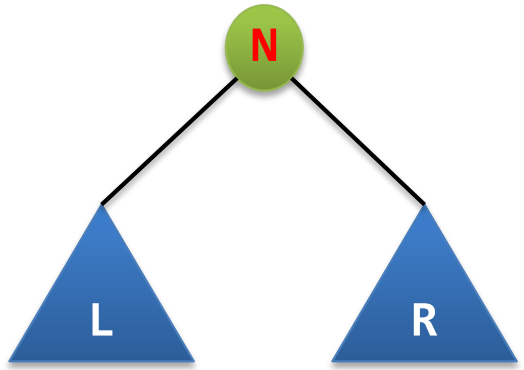
*f*(*b*) ≡ 输出*b*结点　　若*b*为叶子结点

*f*(*b*) ≡ *f*(b->lchild); *f*(b->rchild)　　其他情况

*f*(*b*) ≡ 不做任何事件　　若*b*=NULL
*f*(*b*) ≡ 输出*b*结点　　若*b*为叶子结点
*f*(*b*) ≡ *f*(b->lchild); *f*(b->rchild)　　其他情况

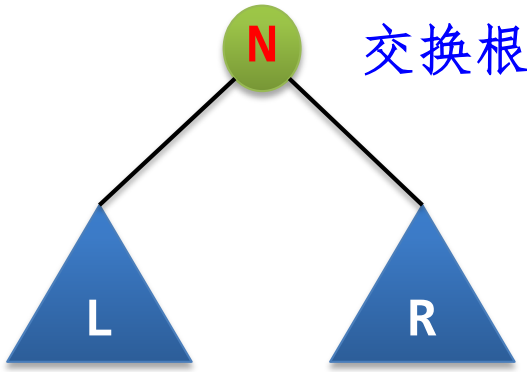
```
void DispLeaf41(BTNode* b)              //基于递归方法输出所有叶子结点
{  if (b!=NULL)
   {  if (b->lchild==NULL && b->rchild==NULL)//根结点为叶子结点时输出
         cout << b->data << " ";
      DispLeaf41(b->lchild);                    //输出左子树的叶子结点
      DispLeaf41(b->rchild);                    //输出右子树的叶子结点
   }
}

void DispLeaf4(BTree& bt)
{
   DispLeaf41(bt.r);
}
```

- 从上述两例看出，基于递归遍历思路和直接采用递归算法设计方法完全相同。实际上，当求解问题较复杂时，直接采用递归算法设计方法更加简单方便。
- 仅从递归遍历角度看，上述两例基于3种递归遍历思路中任意一种都是可行的，但有些情况并非如此。
- 一般地，二叉树由根、左右子树3部分构成，但可以看成两类，即根和子树。
- 如果需要先处理根再处理子树，可以采用先序遍历思路。
- 如果需要先处理子树，再处理根，可以采用后序遍历思路。



【例7.11】假设二叉树中每个结点值为单个字符，采用二叉链存储结构存储。设计一个算法交换二叉树bt的所有左右子树。

递归思路：




交换根结点的左右指针：访问根结点

交换左子树　　交换右子树

采用基于先序遍历的算法：

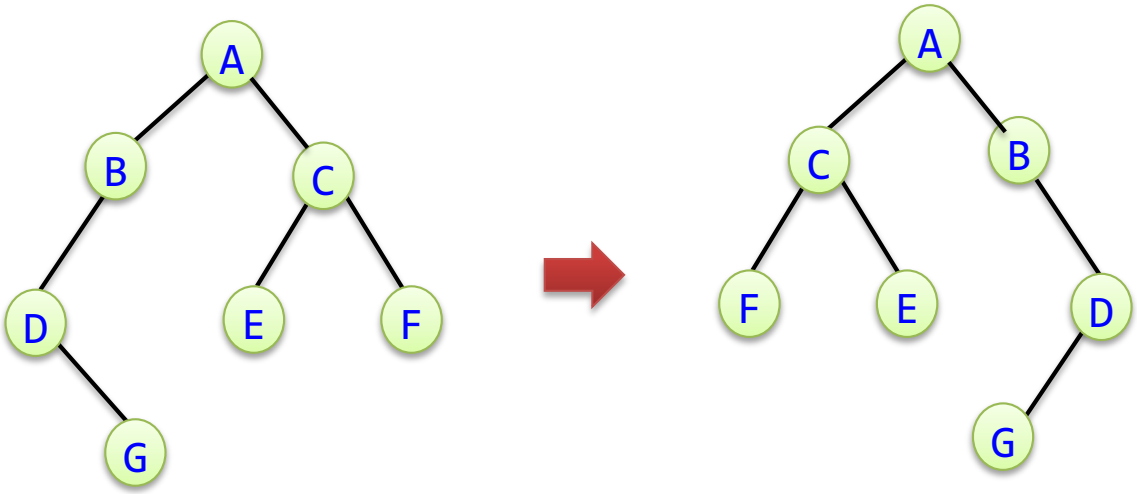
```
void Swap11(BTNode *&b)                    //基于先序遍历
{  if (b!=NULL)
   {  swap(b->lchild,b->rchild);           //交换根结点b的左右孩子指针
      Swap11(b->lchild);                   //递归交换左子树
      Swap11(b->rchild);                   //递归交换右子树
   }
}

void Swap1(BTree &bt)                      //求解算法1
{
   Swap11(bt.r);
}
```

采用基于后序遍历的算法：

```
void Swap21(BTNode *&b)                    //基于后序遍历
{  if (b!=NULL)
   {  Swap21(b->lchild);                   //递归交换左子树
      Swap21(b->rchild);                   //递归交换右子树
      swap(b->lchild,b->rchild);           //交换根结点b的左右孩子指针
   }
}

void Swap2(BTree &bt)                      //求解算法2
{
   Swap21(bt.r);
}
```

程序验证

**采用上述两个算法得到交换结果是正确的。**

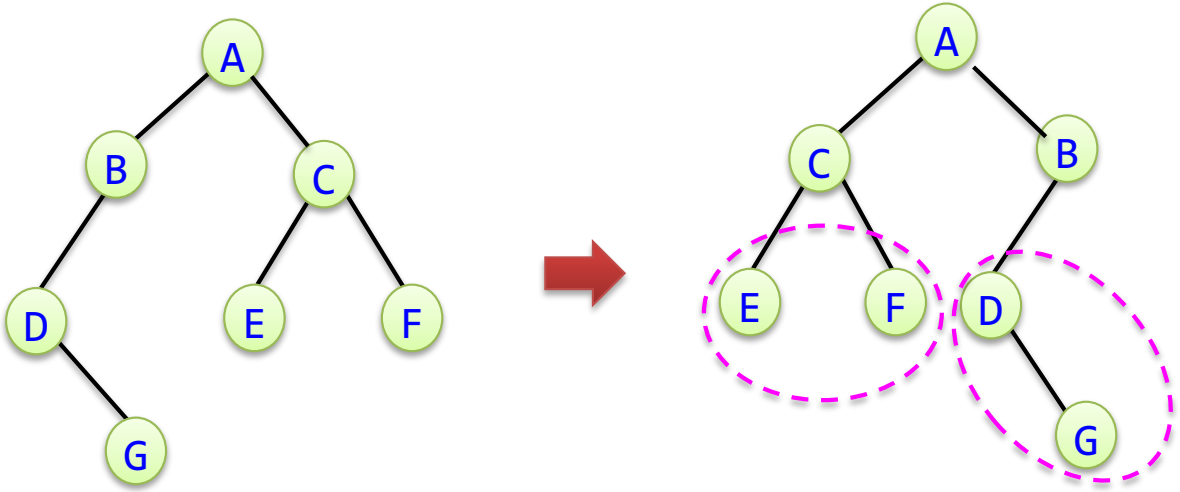


那么**能不能采用中序遍历**呢？对应的基于中序遍历的算法如下：

```
void Swap31(BTNode *&b)                    //基于中序遍历
{  if (b!=NULL)
   {  Swap31(b->lchild);                   //递归交换左子树
      swap(b->lchild,b->rchild);           //交换根结点b的左右孩子指针
      Swap31(b->rchild);                   //递归交换右子树
   }
}

void Swap3(BTree &bt)                      //求解算法3
{
   Swap31(bt.r);
}
```

基于中序遍历算法得到交换结果是错误的！！！

程序验证

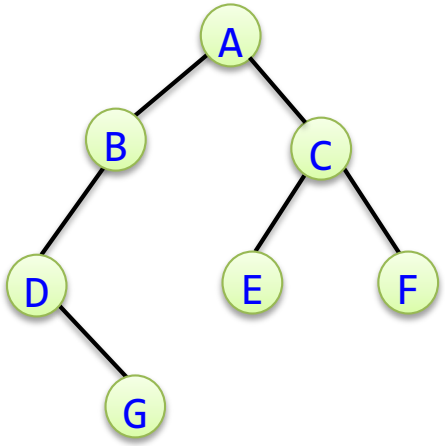



```
void Swap31(BTNode *&b)                  //基于中序遍历
{  if (b!=NULL)
   {  Swap31(b->lchild);                 //递归交换左子树
      swap(b->lchild,b->rchild);         //交换根结点b的左右孩子指针
      Swap31(b->rchild);                 //递归交换右子树
   }
}
```

【例7.12】假设一棵二叉树采用二叉链存储结构，且所有结点值均不相同，设计一个算法求二叉树中指定结点值的结点所在的层次（根结点的层次计为1）。

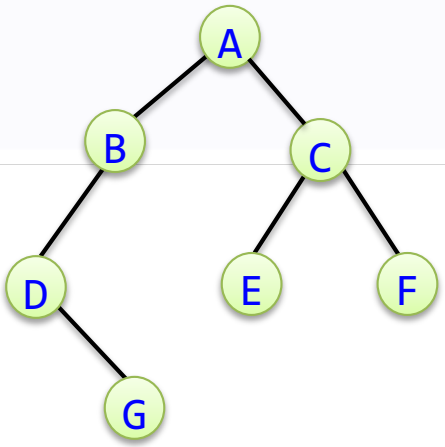
- 二叉树中每个结点都有一个相对于根结点的层次，根结点的层次为1，那么如何指定这种情况呢？
- 可以采用递归算法参数赋初值的方法，即设*f*(*b*，*x*，*h*)为“大问题”，增加第3个参数*h*表示第一个参数*b*指向结点的层次，在初始调用时*b*指向根结点，*h*对应的实参为1，从而指定了根结点的层次为1的情况。

| | |
|---|---|
| $f(b, x, h)=0$ | $b$=NULL |
| $f(b, x, h)=h$ | 当b->data=$x$ |
| $f(b, x, h)=l$ | 当$l=f(b->l\text{child}, x, h+1)$，且$l\neq 0$（在左子树中找到了） |
| $f(b, x, h)=f(b->\text{rchild}, x, h+1)$ | 其他情况 |



```
int Level1(BTNode* b,char x,int h)          //被Level()算法调用
{  if (b==NULL)
      return 0;                             //空树不能找到该结点
   else if (b->data==x)
      return h;                             //根结点即为所找,返回其层次
   else
   {  int l=Level1(b->lchild,x,h+1);        //在左子树中查找
      if (l!=0)                             //左子树中找到了
         return l;                          //返回其层次l
      else                                  //左子树中未找到
         return Level1(b->rchild,x,h+1);    //再在右子树中查找
   }
}

int Level(BTree& bt,char x)                 //求解算法
{
   return Level1(bt.r,x,1);
}
```

**递归算法参数赋初值问题**



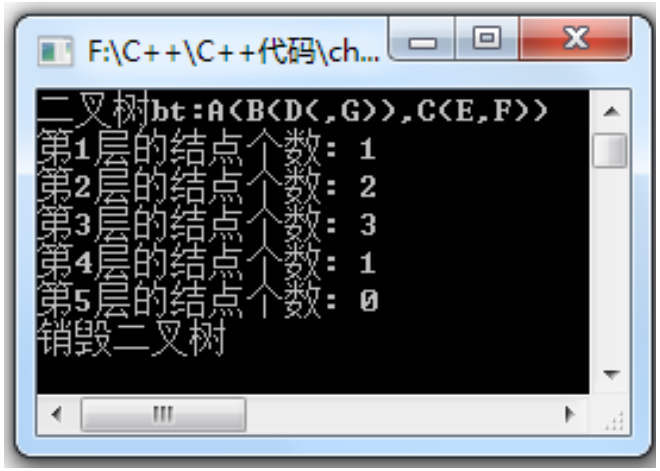
【例7.14】假设二叉树采用二叉链存储结构，且所有结点值均不相同，设计一个算法求二叉树中第$k$（$1 \leqslant k \leqslant$二叉树高度）层的结点个数。

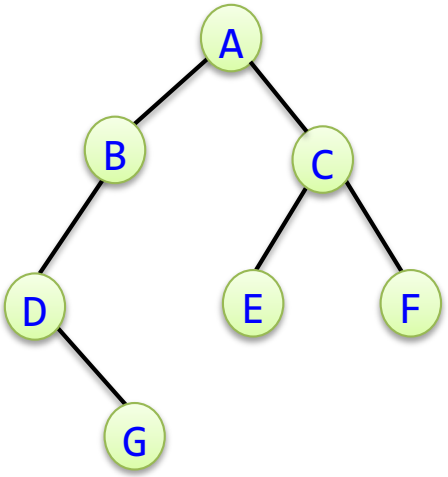
- 采用先序遍历思路。
- 设计KCount1(b，h，k，cnt)递归算法在根结点b的二叉树中求第$k$层的结点个数cnt，其中$h$表示b指向结点的层次（采用参数赋初值方法，b为根结点时，$h$对应的实参数为1）。

```
void KCount1(BTNode* b,int h,int k,int& cnt)
{  if (b==NULL) return;                    //空树返回
   if (h==k) cnt++;                        //当前层的结点在第k层，cnt增1
   if (h<k)                                //当前层次小于k，递归处理求子树
   {  KCount1(b->lchild,h+1,k,cnt);
      KCount1(b->rchild,h+1,k,cnt);
   }
}

int KCount(BTree& bt,int k)                //先序遍历求二叉树第k层结点个数
{  int cnt=0;
   KCount1(bt.r,1,k,cnt);
   return cnt;
}
```

```
int main()
{   string str="A(B(D(,G)),C(E,F))";
    BTree bt;
    bt.CreateBTree(str);
    cout << "二叉树bt:"; bt.DispBTree(); cout << endl;
    for (int k=1;k<=5;k++)
       cout << "第" << k << "层的结点个数: " << KCount(bt,k) << endl;
    cout << "销毁二叉树\n";
    return 0;
}
```

程序验证



【例7.15】假设二叉树采用二叉链存储结构，且所有结点值均不相同，设计一个算法输出值为x的结点的所有祖先。

**解法1**

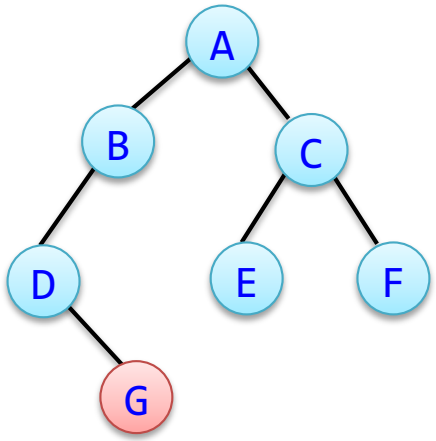
- 根据二叉树中祖先的定义可知，若一个结点的左孩子或右孩子值为x时，则该结点是x结点的祖先结点；若一个结点的左孩子或右孩子为x结点的祖先结点时，则该结点也为x结点的祖先结点。
- 设f(b，x，res)表示b结点是否为结点x的祖先结点，vector<char>容器res存放结点x的祖先结点（从根结点到结点x双亲的结点序列）

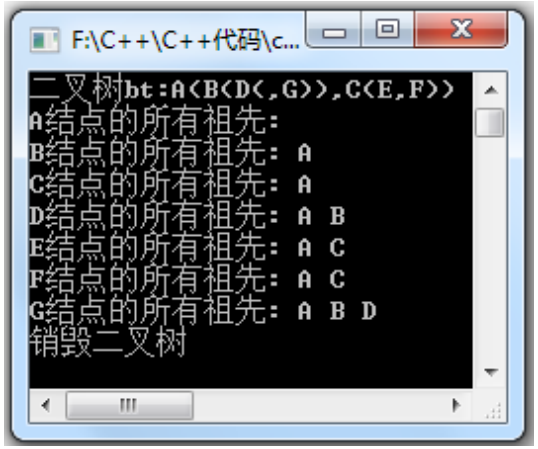
```
f(b，x，res) = false                                  若b==NULL
f(b，x，res) = true; 将b->data添加到res中             若结点b的左孩子为结点x
f(b，x，res) = true，将b->data添加到res中             若结点b的右孩子为结点x
f(b，x，res) = true，将b->data添加到res中
                                          若结点b的左或右孩子是结点x的祖先
f(b，x，res) = false                                  其他情况
```

```
bool Ancestor11(BTNode* b, char x,vector<char>& res)
{  if (b==NULL)                                    //空树返回false
      return false;
   if (b->lchild!=NULL && b->lchild->data==x)
   {  res.push_back(b->data);                      //结点b是x结点的祖先
      return true;
   }
   if (b->rchild!=NULL && b->rchild->data==x)
   {  res.push_back(b->data);                      //结点b是x结点的祖先
      return true;
   }
   if (Ancestor11(b->lchild,x,res) || Ancestor11(b->rchild,x,res))
   {  res.push_back(b->data);  //结点b的孩子是x的祖先，则它是x的祖先
      return true;
   }
   return false;                                   //其他情况返回false
}

void Ancestor1(BTree& bt,char x,vector<char>& res)
 //算法1：返回x结点的祖先
{  Ancestor11(bt.r,x,res);
   reverse(res.begin(),res.end()); //逆置res→按根结点到结点x双亲的顺序
}
```

```
int main()
{   string str="A(B(D(,G)),C(E,F))";
    BTree bt;
    bt.CreateBTree(str);
    cout << "二叉树bt:"; bt.DispBTree(); cout << endl;
    for (char ch='A';ch<='G';ch++)
    {   cout <<  ch << "结点的所有祖先:";
        vector<char> res;                              //存放祖先
        Ancestor1(bt,ch,res);
        for (int i=0;i<res.size();i++)
            cout << " " << res[i];
        cout << endl;
    }
    cout << "销毁二叉树\n";
    return 0;
}
```

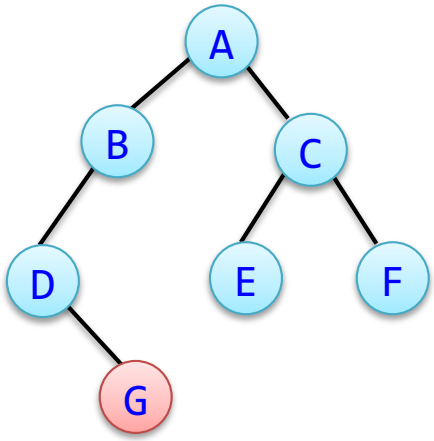
程序验证



## 解法2

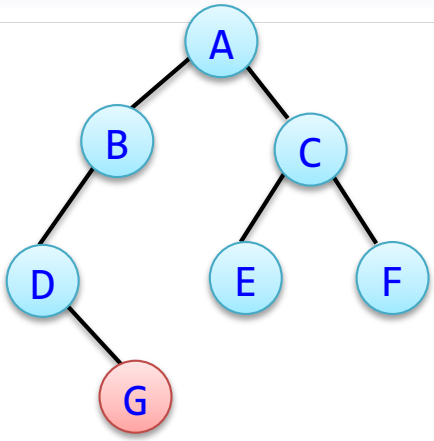
- 二叉树中*x*结点的祖先恰好是根结点到*x*结点的路径上除了*x*结点外的所有结点，用全局变量res向量表示。
- 采用先序遍历的思路，采用一个path向量存放路径，当找到*x*结点时，将path中*x*结点（最后添加的结点）删除，再将path复制的ans中。

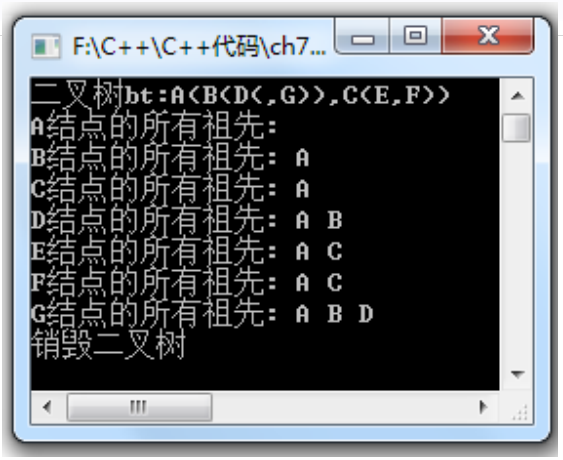



G的祖先：A B D

```
void Ancestor21(BTNode* b,char x,vector<char> path,vector<char>& res)
{  if (b==NULL) return;                          //空树返回
   path.push_back(b->data);
   if (b->data==x)
   {  path.pop_back();                           //删除x结点
      res=path;                                  //复制path到res
      return;                                    //找到路径后返回
   }
   Ancestor21(b->lchild,x,path,res);             //在左子树中查找
   Ancestor21(b->rchild,x,path,res);             //在右子树中查找
}
void Ancestor2(BTree& bt,char x,vector<char>& res)
 //算法2: 返回x结点的祖先
{  vector<char> path;
   Ancestor21(bt.r,x,path,res);
}
```

程序验证

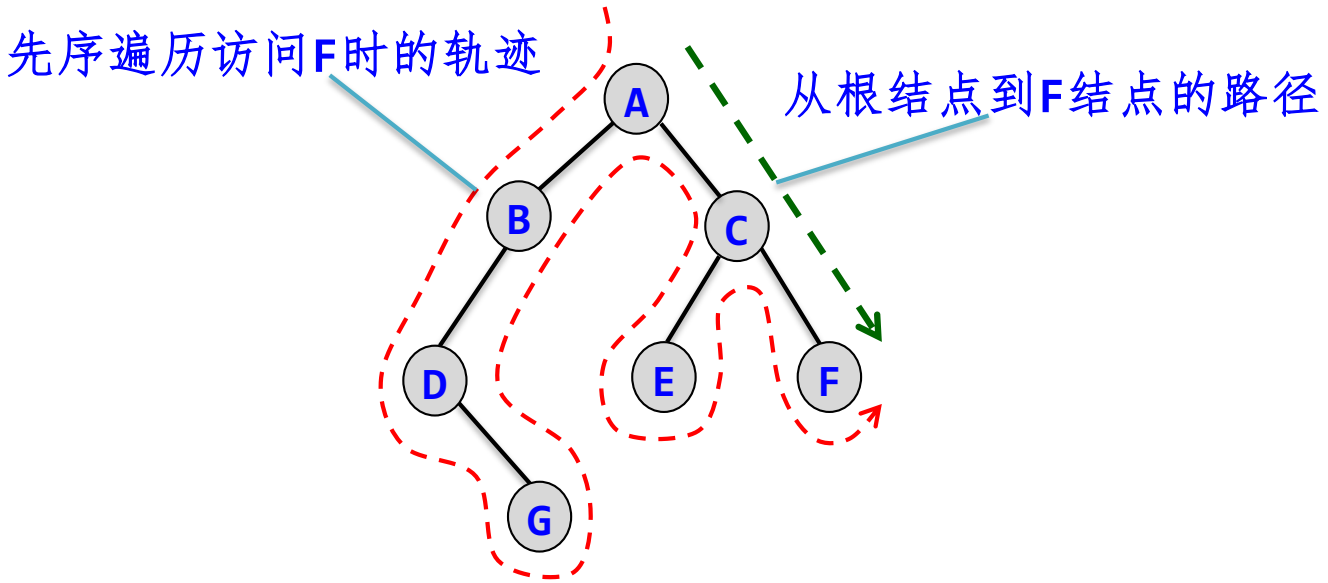


若将path改为引用类型（不用res）返回路径呢

```
Ancestor21(BTNode* b,char x,vector<char> &path)
```

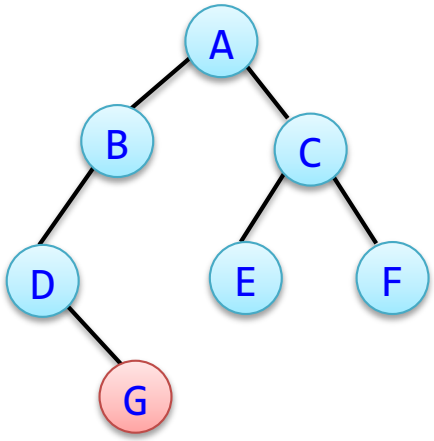
这样在找到x结点时path中存放的是一个查找轨迹（包含所有遍历中访问的结点）

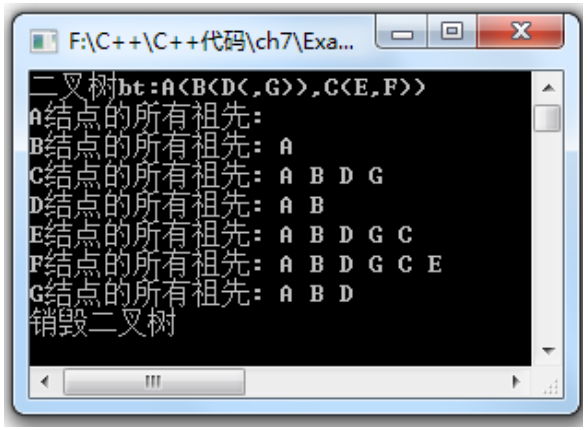


```
void Ancestor21(BTNode* b,char x,vector<char> &path)
{  if (b==NULL) return;
   path.push_back(b->data);
   if (b->data==x)
   {  path.pop_back();                    //删除x结点
      return;                             //找到路径后返回
   }
   Ancestor21(b->lchild,x,path);          //在左子树中查找
   Ancestor21(b->rchild,x,path);          //在右子树中查找
}
void Ancestor2(BTree& bt,char x,vector<char>& res)
//算法2：返回x结点的祖先
{  vector<char> path;
   Ancestor21(bt.r,x,path);
}
```

将path改为引用类型!

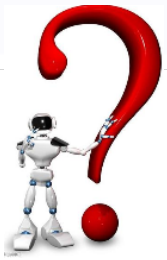
程序验证



进一步改进算法2:

```
bool Ancestor31(BTNode* b,char x,vector<char> path,vector<char>& res)
{  if (b==NULL) return false;                          //空树返回fasle
   path.push_back(b->data);
   if (b->data==x)
   {  path.pop_back();                                 //删除x结点
      res=path;                                        //复制path到res
      return true;                                     //找到后返回true
   }
   if (Ancestor31(b->lchild,x,path,res))               //在左子树中查找
      return true;                                     //左子树中查找成功
   else                                                //左子树中查找失败
      return Ancestor31(b->rchild,x,path,res);         //在右子树中查找
}
void Ancestor3(BTree& bt,char x,vector<char>& res)
//算法3: 返回x结点的祖先
{  vector<char> path;
   Ancestor31(bt.r,x,path,res);
}
```



为什么性能更高?

# 百题筑基

## 二叉树镜像翻转

- 问题描述

  给定一棵二叉树，将其镜像翻转（左右子树交换），然后输出翻转后的先序遍历序列。

- 输入形式

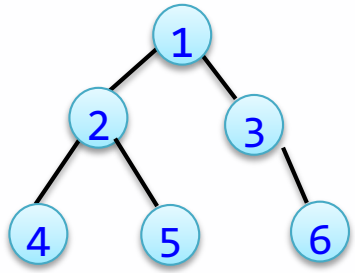
  一行若干个整数，表示二叉树的先序遍历序列，空节点用-1表示。

- 输出形式

  输出镜像翻转后的二叉树的先序遍历序列，空节点不输出。

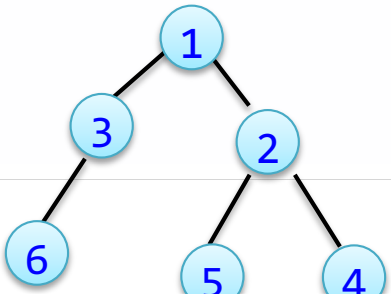
- 样例输入

  **1 2 4 -1 -1 5 -1 -1 3 -1 6 -1 -1**




- 样例输出

  **1 3 6 2 5 4**



# 二叉树先序读入

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

// 递归构建二叉树
TreeNode* buildTree(const std::vector<int>& preorder, int& index) {
    if (index >= preorder.size()) return nullptr;

    int val = preorder[index++];
    if (val == -1) {
        return nullptr;
    }

    TreeNode* node = new TreeNode(val);
    node->left = buildTree(preorder, index);
    node->right = buildTree(preorder, index);
    return node;
}
```

# 二叉树先序读入

```
// 从一行输入解析整数序列
vector<int> readPreorder() {
    string line;
    getline(cin, line);
    istringstream iss(line);
    vector<int> nums;
    int x;
    while (iss >> x) {
        nums.push_back(x);
    }
    return nums;
}

int main() {
    vector<int> preorder = readPreorder();
    int index = 0;
    TreeNode* root = buildTree(preorder, index);

    return 0;
}
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int arr[10010]; int n = 0;
    while (cin >> arr[n]) n++;
    ……

}
```

缺点：

- 固定数组大小风险，越界or浪费；
- 输入结束依赖 EOF
- 无法处理多组独立测试用例

# 7.3.4 先序、中序和后序遍历非递归算法

自学